

▢ PART 1: WHAT IS AN LLM?

1.1 Definition

An **LLM (Large Language Model)** is a deep learning model trained on **billions of words and sentences** to understand and generate human-like language.

It learns **patterns in text**, not fixed rules.

Examples

- ChatGPT
 - Gemini
 - Claude
 - Llama
 - Qwen
 - Mistral
-

1.2 Why is it called "Large"?

Because:

- It is trained on **huge datasets** (books, websites, articles, code, research papers).
- It contains **millions or billions of parameters**.
- It requires massive computational resources to train.

Example:

Model	Approx. Parameters
GPT-2 Small	117 Million
Llama 3 8B	8 Billion
GPT-class models	Billions+

1.3 Core Idea: Next Token Prediction

An LLM predicts **the next most likely token** based on previous tokens.

Example:

```
Input:  
I love eating
```

Prediction:
I love eating biryani.

Generation process:

Prompt:
I love

↓

Predict:
eating

↓

Predict:
biryani

↓

Predict:
every

↓

Predict:
Sunday

↓

Final:
I love eating biryani every Sunday.

The model repeats this process until it decides to stop.

1.4 How Does an LLM Learn?

Training data:

Books
Websites
Wikipedia
Research Papers
Code
Blogs
Conversations

↓

Tokenizer

↓

Tokens

↓

Transformer Neural Network

↓

Predict Next Token

↓

Compare with Correct Answer

↓

Update Weights

↓

Repeat Billions of Times

Eventually, the model learns grammar, reasoning patterns, and language structure.

1.5 Real-World Analogy

Imagine a friend finishes your sentence:

You say:

"The capital of France is..."

They instantly reply:

"Paris."

They didn't search a rulebook—they predicted the most likely continuation based on experience.

An LLM works in a similar way, but using learned statistical patterns from massive text corpora.

1.6 Simple Python Code

```
from transformers import pipeline

generator = pipeline(
    "text-generation",
    model="distilgpt2"
)

result = generator(
    "I love learning AI because",
    max_new_tokens=20
)

print(result[0]["generated_text"])
```

Possible Output

I love learning AI because it allows me to solve complex problems and build intelligent applications.

(Exact output varies because text generation is probabilistic.)

1.7 Explanation of the Code

```
from transformers import pipeline
```

Imports the Hugging Face pipeline utility.

```
pipeline("text-generation")
```

Creates a text generation pipeline.

```
model="distilgpt2"
```

Loads the DistilGPT-2 pretrained language model.

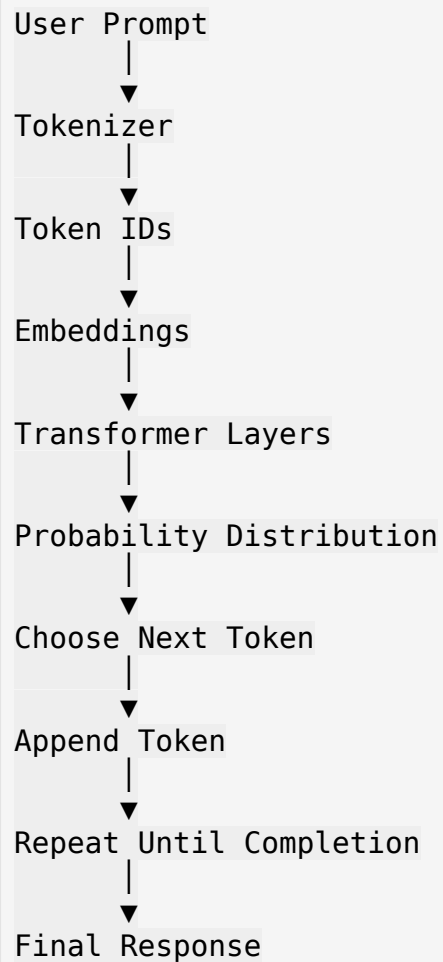
```
max_new_tokens=20
```

Limits the generation to at most 20 newly generated tokens.

```
print(...)
```

Displays the generated text.

1.8 Internal Workflow



1.9 Why Are LLMs Powerful?

They can:

- Answer questions
- Summarize documents
- Translate languages
- Generate code
- Write emails
- Explain concepts
- Assist with research

- Hold conversations
-

1.10 Common Misconceptions

□ "LLMs think like humans."

→ They compute probabilities over sequences of tokens.

□ "LLMs memorize every answer."

→ They learn patterns and representations, though they may memorize some training examples.

□ "LLMs always tell the truth."

→ They can produce incorrect or fabricated information ("hallucinations").

1.11 Interview Questions

Q1: What is an LLM?

Answer: A Large Language Model is a transformer-based neural network trained on large text datasets to predict the next token and generate coherent language.

Q2: Why is it called "Large"?

Answer: Because it is trained on enormous datasets and contains a very large number of learnable parameters.

Q3: What is the primary objective during training?

Answer: Predicting the next token in a sequence.

Q4: Does an LLM store knowledge like a database?

Answer: No. It encodes patterns in its parameters and generates responses based on learned statistical relationships.

□ Key Takeaways

- **LLM = Large Language Model**
- **Built using Transformer architecture**
- **Learns from massive text datasets**

- Generates text by predicting one token at a time
 - Uses tokenization, embeddings, and transformer layers internally
 - Forms the foundation for chatbots, coding assistants, translation systems, and many other AI applications
-

```
from transformers import pipeline
generator=pipeline(
    "text-generation",
    model="distilgpt2"
)
result=generator(
    "I love learning AI because",
    max_new_tokens=20
)
print(result[0]["generated_text"])
```

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

```
{"model_id": "a7c6d56cf8714f418b522eceb9965ff3", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "c2cc65dd3c8a4e8d8affff4cdf24a7118", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "e8adb65442fe4c2fb78d8599b5b09ead", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "4b3b54bf27e54b1294e0b5aa77875f74", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "a838083eed9b40c6806c694887f017cc", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "f91e832c896146b2a0532d1d7dff6aab", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "899edd9f58da430a80e76bb2cf98c00e", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "5ea5dbfc9638419fa818f31069c45a13", "version_major": 2, "version_minor": 0}
```

[transformers] Passing `generation\_config` together with generation-related arguments=({'max_new_tokens'}) is deprecated and will be removed in future versions. Please pass either a `generation\_config`

```
object OR all generation parameters explicitly, but not both.
[transformers] Setting `pad_token_id` to `eos_token_id`:50256 for
open-end generation.
[transformers] Both `max_new_tokens` (=20) and `max_length` (=50) seem
to have been set. `max_new_tokens` will take precedence. Please refer
to the documentation for more information.
(https://huggingface.co/docs/transformers/main/en/main\_classes/text\_ge
neration)
[transformers] Ignoring clean_up_tokenization_spaces=True for BPE
tokenizer GPT2Tokenizer. The clean_up_tokenization post-processing
step is designed for WordPiece tokenizers and is destructive for BPE
(it strips spaces before punctuation). Set
clean_up_tokenization_spaces=False to suppress this warning, or set
clean_up_tokenization_spaces_for_bpe_even_though_it_will_corrupt_outpu
t=True to force cleanup anyway.

I love learning AI because it's so good to have it. I can't understand
why it's so good to have it
```

□ PART 2: Why Traditional Programming Failed (Complete Interview Notes)

Goal: Understand why the world moved from rule-based programming to Machine Learning and finally to Large Language Models (LLMs).

2.1 Introduction

Before AI and LLMs, software was built using **traditional programming**.

In traditional programming:

- Humans write all the rules.
- □ The computer follows those rules exactly.
- □ If a situation isn't covered by a rule, the program fails.

Example

```
Input: "2 + 3"
Rule: Add the numbers
Output: 5
```

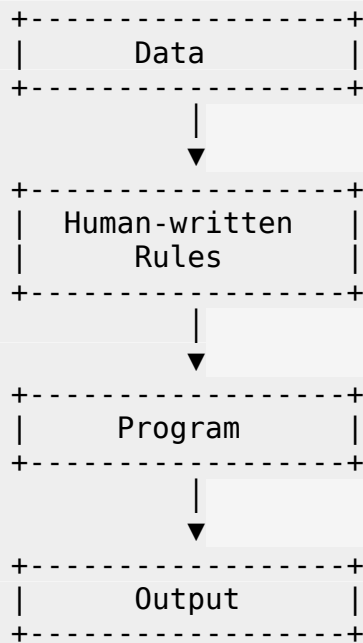
Easy for math.

But what about:

"Is this email spam?" "Translate this paragraph to Telugu." "Write a poem about AI."

Writing rules for these tasks becomes nearly impossible.

2.2 Traditional Programming Workflow



Formula:

Data + Rules = Output

2.3 Python Example (Traditional Programming)

```
def chatbot(user_input):  
    if user_input.lower() == "hello":  
        return "Hi! How can I help you?"  
    elif user_input.lower() == "bye":  
        return "Goodbye!"  
    elif user_input.lower() == "how are you":  
        return "I'm doing great!"  
    else:  
        return "Sorry, I don't understand."  
  
print(chatbot("hello"))
```

```
print(chatbot("bye"))  
print(chatbot("what is AI?"))
```

□ Output

```
Hi! How can I help you?  
Goodbye!  
Sorry, I don't understand.
```

2.4 Line-by-Line Code Explanation

Function definition

```
def chatbot(user_input):
```

Creates a function named `chatbot` that accepts one input string.

First condition

```
if user_input.lower() == "hello":
```

Converts the input to lowercase and checks if it equals `"hello"`.

Return greeting

```
return "Hi! How can I help you?"
```

If the condition matches, the chatbot replies with a greeting.

Second condition

```
elif user_input.lower() == "bye":
```

Checks whether the user typed `"bye"`.

Return goodbye

```
return "Goodbye!"
```

Returns a farewell message.

Third condition

```
elif user_input.lower() == "how are you":
```

Checks for another predefined question.

Return status

```
return "I'm doing great!"
```

Responds with a fixed answer.

Default case

```
else:  
    return "Sorry, I don't understand."
```

If none of the predefined rules match, the chatbot cannot answer.

Function calls

```
print(chatbot("hello"))
```

Executes the function with "hello" and prints the result.

2.5 Problems with Traditional Programming

□ Problem 1: Too Many Rules

Suppose you want to recognize greetings.

People may type:

```
Hello  
hello  
HELLO  
Hi
```

```
hi
Hey
hey there
Good morning
Good evening
```

You would need to write a rule for every variation.

□ Problem 2: Doesn't Learn

If a user says:

```
Howdy!
```

The program won't understand unless you manually add another rule.

□ Problem 3: Doesn't Understand Meaning

Example:

```
I am happy today.
```

and

```
Today I feel joyful.
```

Both have similar meanings, but rule-based systems often treat them as completely different.

□ Problem 4: Hard to Scale

Imagine writing rules for:

- Every language
- Every spelling mistake
- Every synonym
- Every grammar variation

This quickly becomes unmanageable.

2.6 Real-World Analogy

Imagine a security guard with a checklist:

```
IF person wears red shirt → Allow  
IF person wears blue shirt → Allow  
ELSE → Stop
```

Someone wearing a green shirt arrives.

The guard doesn't know what to do.

An LLM is more like an experienced human who understands context instead of relying only on fixed rules.

2.7 How Machine Learning Changed the Formula

Instead of writing rules manually:

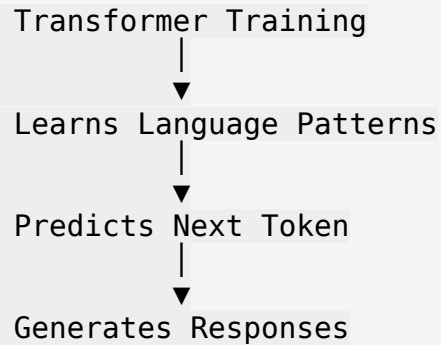
```
Data + Desired Outputs  
  ↓  
Machine Learning  
  ↓  
Learns the Rules Automatically
```

Formula:

```
Data + Answers → Model learns Rules
```

2.8 How LLMs Improved Further

```
Billions of Documents  
  ↓  
Tokenization  
  ↓
```



No human writes explicit grammar rules or conversation rules.

2.9 Traditional Programming vs LLM

Traditional Programming	LLM
Human writes rules	Learns patterns from data
Fixed responses	Flexible generation
Cannot generalize well	Generalizes to unseen inputs
Difficult to maintain	Scales through training
Fails on unknown cases	Often handles variations naturally

2.10 Example: Translation

Traditional approach

```
IF "Hello" → "Hola"  
IF "Thank you" → "Gracias"  
IF "Good morning" → "Buenos días"  
...
```

Thousands or millions of rules are required.

LLM approach

Prompt:

```
Translate to Spanish:  
"Good morning, how are you?"
```

Output:

```
"Buenos días, ¿cómo estás?"
```

The model generates the translation without hard-coded phrase mappings.

2.11 Why LLMs Are Better

LLMs can:

- Understand context
- Handle synonyms
- Generate new sentences
- Adapt to many writing styles
- Perform multiple tasks with the same model

Examples:

- Chatbots
 - Code generation
 - Translation
 - Summarization
 - Story writing
 - Email drafting
-

2.12 Mini Hugging Face Example

```
from transformers import pipeline
generator = pipeline("text-generation", model="distilgpt2")
prompt = "Artificial Intelligence will"
result = generator(prompt, max_new_tokens=15)
print(result[0]["generated_text"])
```

Possible Output

```
Artificial Intelligence will continue to transform healthcare,
education, and scientific research.
```

Explanation

```
pipeline("text-generation", model="distilgpt2")
```

Loads a pretrained GPT-style model for text generation.

```
prompt = "Artificial Intelligence will"
```

Provides the starting text.

```
generator(prompt, max_new_tokens=15)
```

Asks the model to generate up to 15 additional tokens.

```
print(result[0]["generated_text"])
```

Prints the completed text.

2.13 Interview Questions

Q1. Why did traditional programming struggle with language tasks?

Answer: Because natural language is highly variable and requires an impractical number of manually written rules.

Q2. What is the main difference between traditional programming and LLMs?

Answer: Traditional programming follows explicit rules written by developers, while LLMs learn statistical patterns from large datasets.

Q3. Can traditional programs learn from new examples automatically?

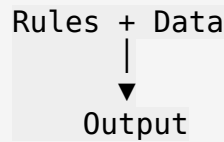
Answer: No. They require developers to update or add rules manually.

Q4. Why are LLMs more flexible?

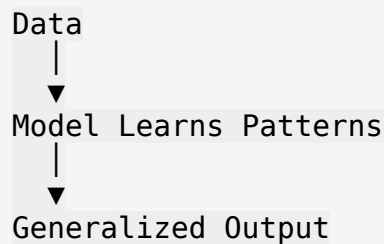
Answer: They generalize from learned patterns and can produce appropriate responses even for inputs they have never seen exactly before.

□ Memory Trick

Traditional Programming



Machine Learning / LLMs



□ Key Takeaways

- Traditional programming depends on **explicit human-written rules**.
- It works well for structured tasks but poorly for open-ended language understanding.
- LLMs replace handcrafted rules with **learning from massive datasets**.
- The transition from rule-based systems to learning-based models is a key reason modern AI can perform translation, summarization, coding assistance, and conversational tasks effectively.

bold text

```
def chatbot(user_input):  
    if user_input.lower()=="hello":  
        return "Hi ! How can I help you?"  
    elif user_input.lower()=="bye":  
        return "Goodbye!"  
    elif user_input.lower()=="how are you":  
        return "I'm doing great!"  
    else:  
        return "Sorry , I don't understand."  
  
print(chatbot("hello"))  
print(chatbot("bye"))  
print(chatbot("what is AI"))
```

Hi ! How can I help you?
Goodbye!
Sorry , I don't understand.

2.12 Mini Hugging Face Example

```
from transformers import pipeline
generator=pipeline("text-generation",model="distilgpt2")
prompt="Artificial Intelligence will"
result=generator(prompt,max_new_tokens=15)
print(result[0]["generated_text"])

{"model_id":"e7e2047a2bf64f61a0cdd3b14ea43587","version_major":2,"version_minor":0}
```

```
[transformers] Setting `pad_token_id` to `eos_token_id`:50256 for
open-end generation.
[transformers] Both `max_new_tokens` (=15) and `max_length` (=50) seem
to have been set. `max_new_tokens` will take precedence. Please refer
to the documentation for more information.
(https://huggingface.co/docs/transformers/main/en/main\_classes/text\_generation)
```

Artificial Intelligence will always be the same.

My last post was about the game,

□ PART 3: What Are Tokens? (Complete Interview Notes)

Goal: Understand what tokens are, why LLMs use them instead of words, how tokenization works, and how to use Hugging Face tokenizers with code.

3.1 What is a Token?

A **token** is the **smallest unit of text** that an LLM processes.

A token **is not always a word**. It can be:

- A complete word
- Part of a word (subword)
- A single character
- A punctuation mark
- A number

- A special symbol

Example

Sentence:

I love ChatGPT!

Possible Tokens:

["I", "love", "Chat", "G", "PT", "!"]

The exact split depends on the tokenizer used.

3.2 Why Don't LLMs Process Whole Words?

Suppose an LLM only stored complete words.

It would need separate entries for:

```
run
running
runner
runs
ran
```

Instead, it can split them into reusable pieces:

```
run + ning
run + ner
run + s
```

This:

- □ Reduces vocabulary size
 - □ Handles unknown words better
 - □ Improves efficiency
-

3.3 Real-Life Analogy

Think of **LEGO blocks**.

Instead of storing every possible building, you store reusable blocks and combine them.

Similarly:

```
un + happy  
play + ing  
read + able
```

The model combines smaller pieces to understand new words.

3.4 Example Tokenization

Sentence:

```
I love machine learning.
```

Possible tokens:

```
["I", "love", "machine", "learning", "."]
```

Another example:

```
unbelievable
```

May become:

```
["un", "believ", "able"]
```

3.5 Types of Tokenization

A. Character Tokenization

```
Hello
```

```
↓
```

```
H  
e  
l  
l  
o
```

Advantages:

- Small vocabulary

Disadvantages:

- Very long sequences
-

B. Word Tokenization

Hello world today

↓

["Hello", "world", "today"]

Advantages:

- Easy to understand

Disadvantages:

- Unknown words become difficult.
-

C. Subword Tokenization □ (Used by Modern LLMs)

unbelievable

↓

["un", "believ", "able"]

Advantages:

- Handles unseen words
 - Smaller vocabulary
 - Better generalization
-

3.6 Special Tokens

Many models use reserved tokens.

Examples:

<BOS>	Beginning of sentence
<EOS>	End of sentence
<PAD>	Padding
<UNK>	Unknown token
<MASK>	Masked word (BERT)

Example:

```
<BOS> Hello world <EOS>
```

3.7 Token IDs

Computers cannot process text directly.

Each token gets converted into an integer.

Example:

Token	ID
Hello	15496
world	995
!	0

The model actually processes:

```
[15496, 995, 0]
```

3.8 Hugging Face Tokenizer Code

```
from transformers import AutoTokenizer

# Load GPT-2 tokenizer
tokenizer = AutoTokenizer.from_pretrained("gpt2")

text = "I love artificial intelligence."

# Convert text into tokens
tokens = tokenizer.tokenize(text)

# Convert tokens into IDs
token_ids = tokenizer.encode(text)

print("Tokens:")
print(tokens)

print("\nToken IDs:")
print(token_ids)
```

3.9 Possible Output

```
Tokens:  
['I', 'Ġlove', 'Ġartificial', 'Ġintelligence', '.']  
  
Token IDs:  
[40, 1842, 11666, 4430, 13]
```

Note: GPT-2 uses `Ġ` to indicate a leading space before a token.

3.10 Line-by-Line Code Explanation

Import

```
from transformers import AutoTokenizer
```

Imports Hugging Face's tokenizer utility.

Load tokenizer

```
tokenizer = AutoTokenizer.from_pretrained("gpt2")
```

Downloads and loads GPT-2's tokenizer.

It contains:

- Vocabulary
 - Tokenization rules
 - Token-to-ID mappings
-

Input text

```
text = "I love artificial intelligence."
```

Defines the input sentence.

Tokenize

```
tokens = tokenizer.tokenize(text)
```

Splits text into model-recognized tokens.

Possible result:

```
['I', 'love', 'artificial', 'intelligence', '.']
```

Encode

```
token_ids = tokenizer.encode(text)
```

Converts tokens into integer IDs.

Example:

```
[40, 1842, 11666, 4430, 13]
```

Print

```
print(tokens)
print(token_ids)
```

Displays both tokens and their numeric IDs.

3.11 Decode Back to Text

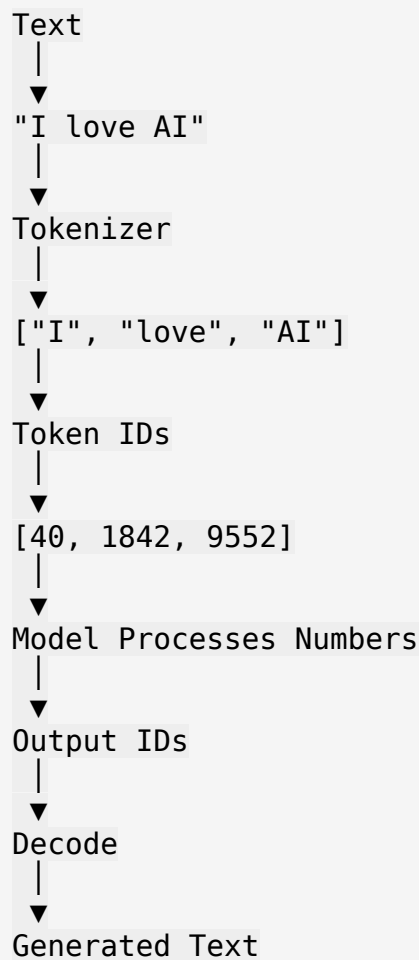
```
decoded = tokenizer.decode(token_ids)
print(decoded)
```

Output

```
I love artificial intelligence.
```

This converts IDs back into readable text.

3.12 Complete Encode → Decode Flow



3.13 Why Tokens Are Important

Without tokenization:

- Computers cannot understand text.
- Neural networks require numeric inputs.

Tokens are the bridge between **human language** and **machine computation**.

3.14 Context Window Uses Tokens

Suppose a model has:

Context Window = 8 tokens

Input:

I love learning about artificial intelligence every single day.

The tokenizer might produce 10+ tokens.

If the context limit is exceeded:

- Older tokens may be dropped, or
- The input may be truncated or chunked.

This is why context windows are measured in **tokens**, not characters or words.

3.15 Interview Questions

Q1. What is a token?

A token is the smallest text unit processed by an LLM. It may be a word, subword, punctuation mark, or symbol.

Q2. Why don't LLMs use words directly?

Because subword tokenization reduces vocabulary size and allows models to handle unseen words efficiently.

Q3. What is tokenization?

The process of converting raw text into tokens before feeding it into the model.

Q4. What is a token ID?

A numeric representation assigned to each token so that neural networks can process it.

Q5. What does `AutoTokenizer` do?

It loads the tokenizer associated with a pretrained model and provides methods to tokenize, encode, and decode text.

Q6. Why do GPT-2 tokens sometimes start with `Ġ`?

The `Ġ` symbol indicates that the token begins after a space, helping the tokenizer preserve whitespace information.

3.16 Common Mistakes

❑ 1 word = 1 token

Wrong. A word can split into multiple tokens.

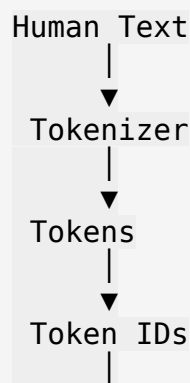
❑ Characters equal tokens

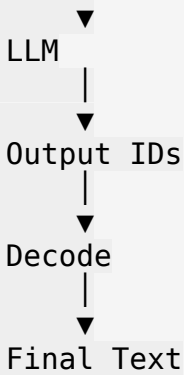
Wrong. Modern LLMs mostly use **subword tokenization**.

❑ All tokenizers behave the same

Wrong. GPT, Llama, BERT, and other models use different vocabularies and tokenization strategies.

3.17 Memory Trick





□ Key Takeaways

- □ A **token** is the basic unit processed by an LLM.
- □ Tokens can be **words, subwords, punctuation, or symbols**.
- □ Modern LLMs primarily use **subword tokenization**.
- □ Each token is mapped to a **numeric ID** before entering the neural network.
- □ `AutoTokenizer` is used to tokenize, encode, and decode text.
- □ Context windows and model inputs are measured in **tokens**, not words.

□ Interview Tip

If asked:

"Does ChatGPT read sentences word by word?"

A strong answer is:

"No. ChatGPT first tokenizes the input into subword tokens, converts them to token IDs, transforms them into embeddings, and then processes them through Transformer layers to predict the next token."

This level of explanation demonstrates a solid understanding of LLM internals. **bold text**

```
from transformers import AutoTokenizer
tokenizer=AutoTokenizer.from_pretrained("gpt2")
text="I love Artificial Intelligence."
tokens=tokenizer.tokenize(text)
token_ids=tokenizer.encode(text)
print("Tokens:")
print(tokens)
print("\n Token IDs:")
print(token_ids)
```

```
{"model_id":"79097be158634033bee9ebce0e381e6a","version_major":2,"version_minor":0}
```

```
{"model_id":"6856684c6f454f6a95edb7c647ab96ca","version_major":2,"version_minor":0}
```

```
{"model_id":"df1f7b3b211a48e4aad18e4764a7e1e6","version_major":2,"version_minor":0}
```

```
{"model_id":"d40e058685544f11bd89aa817a08fbe1","version_major":2,"version_minor":0}
```

```
{"model_id":"0f9e089b98d14e11bee17da4d8962081","version_major":2,"version_minor":0}
```

Tokens:

```
['I', 'Glove', 'GArt', 'if', 'ical', 'GIntelligence', '.']
```

Token IDs:

```
[40, 1842, 3683, 361, 605, 9345, 13]
```

```
from sentence_transformers import SentenceTransformer
```

```
# Load pretrained embedding model
```

```
model = SentenceTransformer(  
    "sentence-transformers/all-MiniLM-L6-v2"  
)
```

```
sentence = "I love Artificial Intelligence"
```

```
embedding = model.encode(sentence)
```

```
print(embedding)  
print(len(embedding))
```

```
{"model_id":"15475fb00d544ee69f0334250c63ac28","version_major":2,"version_minor":0}
```

```
{"model_id":"e2ca94134071417784e2d875d90a4a40","version_major":2,"version_minor":0}
```

```
{"model_id":"13806d3887cd4d57b7c97ef5be33c80a","version_major":2,"version_minor":0}
```

```
{"model_id":"bd4d1b95b6344521b1a382faeaacf835","version_major":2,"version_minor":0}
```

```
{"model_id":"d638ee8fc99a454f87c58bfe22581b75","version_major":2,"version_minor":0}
```

```
{"model_id":"9b8c6c4de651458ca851e628921fc4e6","version_major":2,"version_minor":0}
```

```
{"model_id":"e43d6efc07ed46aa856b18fde6ad2ef4","version_major":2,"version_minor":0}
```

```
{"model_id":"dd1166b9b0384c7a9101980d2ede2bad","version_major":2,"version_minor":0}
```

```
{"model_id":"468c8e52d7674bdca008013d41c0bdaf","version_major":2,"version_minor":0}
```

```
{"model_id":"43c2562a4d93474dafef53d9de112e97","version_major":2,"version_minor":0}
```

```
{"model_id":"f39554bc0c864ccfaaf2f970bc3e9269","version_major":2,"version_minor":0}
```

```
{"model_id":"9dc6d75aadb34a578d85e3a0c04ec18f","version_major":2,"version_minor":0}
```

```
[-2.61359382e-02 -6.41852394e-02  5.79342693e-02 -2.27682739e-02
 3.99345234e-02 -6.94327578e-02  5.09679466e-02  2.81137656e-02
 5.69300391e-02  4.83488627e-02 -4.29284051e-02  6.80153072e-03
 3.28429788e-03  4.65278104e-02 -3.27342339e-02  2.20695119e-02
-4.11613174e-02 -1.76253524e-02 -1.33416325e-01 -1.12813227e-01
-9.07842293e-02  4.84751761e-02  2.20636055e-02 -4.63822745e-02
-2.57994384e-02  3.99871618e-02  2.86491704e-03 -6.62114248e-02
-8.65889993e-03 -8.70668218e-02 -2.54891044e-03  3.31529118e-02
 7.47496933e-02  3.48976551e-04 -6.45886958e-02  5.41341826e-02
 3.39243002e-03 -1.88701507e-02  6.48412257e-02  2.78316118e-04
-3.98724973e-02 -1.17227165e-02  5.26453517e-02 -5.42396959e-03
 4.97092642e-02  7.94329420e-02 -7.28190318e-02 -4.01672609e-02
 7.00027421e-02  4.89695184e-02 -8.06456506e-02 -6.57430245e-03
-2.13147737e-02  8.11341777e-03  6.76508294e-03  2.71840505e-02
 5.28748855e-02  1.98687166e-02 -2.27795094e-02 -6.46123812e-02
 2.74949465e-02 -2.05367198e-03  3.96205150e-02  1.41450856e-03
 6.55848980e-02  3.44113074e-02 -1.92249306e-02 -1.29991788e-02
 2.09078658e-02 -2.10647434e-02 -1.42407510e-03  1.19073942e-01
 4.86778617e-02  4.52229343e-02  6.85076639e-02  1.21972170e-02
 3.87232751e-02 -9.38269589e-03  8.96642283e-02 -1.24514811e-02
 1.00642825e-02 -1.04419693e-01 -3.58741619e-02  4.40047793e-02
 3.67982350e-02 -1.04027808e-01 -3.20286751e-02  3.03936154e-02
-1.26120970e-02 -1.15427366e-02 -3.01279267e-03 -4.21703011e-02
-2.92378534e-02 -4.70341705e-02 -6.59344625e-03  2.01082672e-03
-7.06236297e-03 -8.21972638e-02 -6.42996654e-02  7.96739906e-02
-7.21231475e-02 -3.57368239e-03 -1.85819287e-02  5.66394851e-02
-1.04288859e-02  2.02330034e-02  4.61944984e-03  1.26741100e-02
 1.04482859e-01 -1.03396811e-01 -4.34212312e-02  2.97888294e-02
 7.77221099e-02 -5.38481213e-02 -7.39479205e-03  4.09289077e-02
 3.97238322e-02  1.16576627e-01  8.78579170e-02  8.52997694e-03
-2.59399158e-03  5.14718555e-02 -5.00769280e-02  7.68050551e-02
 2.76088882e-02 -7.14803487e-02 -4.64265682e-02 -3.42503238e-33
-1.32470936e-01 -2.33422592e-02  5.77494875e-02  2.14310847e-02
```

5.41784661e-03	-6.59241453e-02	3.91480215e-02	-1.43231666e-02
-5.09760156e-02	-2.02047848e-03	-7.67177194e-02	5.72049096e-02
-5.87485433e-02	7.61633143e-02	5.04899025e-02	-3.89729701e-02
-2.96697523e-02	2.26469580e-02	-5.05912267e-02	-3.25429551e-02
5.91508560e-02	-5.21261292e-03	-4.91590053e-03	2.23276392e-02
7.60436477e-03	8.26065312e-04	5.04796505e-02	-8.81623849e-02
5.21589518e-02	2.65464410e-02	-2.83843614e-02	8.32826719e-02
-5.43997176e-02	-2.10154895e-03	-1.76239423e-02	-3.66027951e-02
-3.75857800e-02	9.07489657e-03	2.83470713e-02	8.75845402e-02
2.29353085e-02	1.58738587e-02	5.98291215e-03	6.60182722e-03
6.23015836e-02	-1.11340154e-02	7.42245317e-02	1.22902896e-02
5.56281656e-02	-6.21796921e-02	-6.86704889e-02	2.12240461e-02
3.84100489e-02	-2.23451406e-02	1.04583807e-01	-7.82319345e-03
8.12094659e-03	9.96278413e-03	-3.27920988e-02	-1.04859564e-02
-3.37117612e-02	5.42346239e-02	-3.48232198e-03	6.68048188e-02
1.06420498e-02	4.88838367e-02	2.83260643e-03	3.27085517e-02
7.16905594e-02	4.52011153e-02	-3.55790369e-02	3.45349051e-02
-2.10510492e-02	-3.92735340e-02	-9.09156501e-02	-3.29045057e-02
7.03085512e-02	-1.11507893e-01	-1.74872167e-02	-2.71691494e-02
-6.12120777e-02	-5.81883565e-02	-2.09212024e-02	-6.47916570e-02
1.03414081e-01	2.49599982e-02	-1.95037797e-02	-9.03077573e-02
1.86033715e-02	1.78855052e-03	-5.43465232e-03	-9.32209939e-02
4.64217030e-02	-7.63579388e-04	-1.18307538e-01	3.38511553e-33
-8.07337388e-02	-4.59681153e-02	-4.42557819e-02	7.18273148e-02
-2.24945359e-02	-4.63295393e-02	-7.89525881e-02	1.02658989e-02
1.47407223e-02	1.39019847e-01	3.23939472e-02	-2.00785007e-02
-4.21140296e-03	4.37668599e-02	-5.61739551e-03	1.09174345e-02
-2.95942705e-02	-2.21585166e-02	-1.32482499e-02	3.16727795e-02
-1.26731014e-02	8.52391198e-02	-8.39328170e-02	-2.68443990e-02
1.31567428e-02	5.11741005e-02	-5.43063581e-02	3.99026787e-03
3.07765398e-02	6.77290037e-02	2.35621650e-02	-3.23609933e-02
-6.15347736e-02	6.44269679e-03	4.04248154e-03	2.73840018e-02
-1.84226893e-02	-5.04739620e-02	-1.38327740e-02	7.29411887e-03
3.07853725e-02	-3.11256223e-03	-1.59155601e-03	2.35129911e-02
3.39228548e-02	-6.45082095e-04	1.66966747e-02	4.90378551e-02
-2.17952691e-02	-3.03462148e-02	-7.64985457e-02	-2.62126736e-02
-3.31651084e-02	-1.60428166e-01	-3.24510150e-02	2.50137839e-02
6.17458671e-02	2.42848881e-02	-3.39357927e-02	3.03807817e-02
-4.50252555e-02	-3.65880318e-02	5.47638908e-02	5.49614690e-02
-8.43113381e-03	-2.10654810e-02	2.15938520e-02	2.98942882e-03
-7.81012923e-02	-9.30328220e-02	3.63332517e-02	3.79475113e-03
-3.14266905e-02	6.03360906e-02	-9.82008595e-03	-2.19497718e-02
-2.21983232e-02	5.58994412e-02	-2.19993815e-02	-2.95621506e-03
5.05715460e-02	-2.47612037e-02	1.60027724e-02	1.05632722e-01
-3.89018245e-02	1.07959487e-01	-1.54263629e-02	-1.66745745e-02
-4.84121516e-02	-3.57722081e-02	-1.76417232e-02	7.56726488e-02
-9.40129452e-04	-2.33332813e-02	-5.06527498e-02	-1.36772833e-08
1.02954879e-02	-4.38677240e-03	1.05834752e-01	-8.37391689e-02
5.12444489e-02	2.20832182e-03	-3.28079313e-02	-2.19304655e-02

```
-9.29033682e-02 -2.86644492e-02 5.65826334e-02 3.36488970e-02
1.00692278e-02 4.70551103e-02 1.14811301e-01 6.75453432e-03
6.92354143e-02 -4.33608033e-02 1.59705002e-02 4.60113622e-02
3.79140079e-02 4.30767909e-02 -1.91264041e-02 2.00283434e-02
-5.01580313e-02 -1.08623756e-02 -6.21110853e-03 -1.70872677e-02
-1.19180670e-02 6.59174919e-02 -3.15025337e-02 3.74695435e-02
-1.38822626e-02 4.16132957e-02 7.91253671e-02 7.42028058e-02
1.40718007e-02 -7.79613480e-02 -1.57327522e-02 -6.61861748e-02
5.05652232e-03 1.11994661e-01 -3.53579372e-02 -4.92915250e-02
3.89640145e-02 5.22447610e-03 4.13429104e-02 -1.28480792e-01
5.92310242e-02 4.96794991e-02 -5.81577420e-03 3.26042883e-02
2.03441512e-02 1.04296200e-01 1.45285442e-01 1.05475727e-02
-1.38585176e-03 -3.95541191e-02 -1.72352307e-02 1.04589298e-01
5.90290688e-02 9.32972431e-02 3.74128744e-02 -1.11665227e-01]
```

384

□ PART 4: What Are Embeddings? (Complete Interview Notes)

Goal: Understand what embeddings are, why LLMs need them, how they are created, and how they help models understand the meaning of text.

4.1 What Are Embeddings?

An **embedding** is a **numerical vector (list of decimal numbers)** that represents the meaning of a word, sentence, or document.

□ In simple words:

Embeddings convert human language into numbers that AI models can understand while preserving semantic meaning.

Example

Word: "Cat"

Embedding:

[0.23, -0.91, 0.45, 0.78, ..., 0.12]

The model **does not understand the word "Cat" directly**. It understands its **embedding vector**.

4.2 Why Do We Need Embeddings?

Neural networks **cannot process text directly**.

Example:

```
"I love AI"
```

The computer cannot calculate with letters.

So we convert it into:

```
"I love AI"
  ↓
Tokenization
  ↓
["I", "love", "AI"]
  ↓
Token IDs
  ↓
[40, 1842, 9552]
  ↓
Embeddings
  ↓
[
  [0.12, 0.45, ...],
  [0.91, -0.21, ...],
  [0.67, 0.10, ...]
]
```

Now the Transformer can process the input mathematically.

4.3 Real-Life Analogy

Imagine a world map.

```
Hyderabad  ----  Bangalore
      \
```

\
Chennai

Cities that are geographically close are near each other on the map.

Similarly, in embedding space:

King ----- Queen

Man ----- Woman

Cat ----- Dog

Apple ----- Mango

Words with similar meanings are located close together.

4.4 Difference Between Token IDs and Embeddings

Token ID	Embedding
Integer	Floating-point vector
Example: 9552	Example: [0.34, -0.72, ...]
Just an index	Contains semantic meaning
Used to look up vectors	Used by the Transformer

4.5 Example

Sentence:

I love machine learning.

After tokenization:

["I", "love", "machine", "learning"]

Token IDs:

[40, 1842, 4572, 8932]

Embeddings:

```
[  
  [0.12, 0.44, -0.31],  
  [0.91, 0.17, 0.52],  
  [0.77, 0.29, -0.18],  
  [0.83, 0.81, 0.22]  
]
```

The Transformer uses these vectors—not the original words.

4.6 Embedding Dimensions

An embedding is usually high-dimensional.

Examples:

Model	Embedding Size
MiniLM	384
BERT Base	768
BERT Large	1024
Some LLMs	2048 or higher

Example:

Embedding (384 dimensions)

```
[  
  0.12,  
 -0.54,  
  0.88,  
  ...  
  384 values  
]
```

4.7 Similar Words Have Similar Embeddings

Example:

```
King  
Queen
```

Prince
Princess

These vectors are close together.

But:

King
Laptop
Pizza

Their vectors are far apart.

This property enables semantic understanding.

4.8 Word Embeddings vs Sentence Embeddings

Word Embedding

Input:

Dog

Output:

[0.12, 0.44, ...]

Sentence Embedding

Input:

Dogs are loyal animals.

Output:

[0.81, -0.42, 0.11, ...]

Sentence embeddings capture the meaning of the **entire sentence**, not just one word.

4.9 Python Example (SentenceTransformer)

```
from sentence_transformers import SentenceTransformer

# Load pretrained embedding model
model = SentenceTransformer(
    "sentence-transformers/all-MiniLM-L6-v2"
)

sentence = "I love Artificial Intelligence"

embedding = model.encode(sentence)

print(embedding)
print(len(embedding))
```

4.10 Expected Output

```
[ 0.081,
 -0.132,
  0.541,
  ...,
  0.223]

384
```

The numbers will differ each run/model, but the length is typically **384** for this model.

4.11 Line-by-Line Code Explanation

Import

```
from sentence_transformers import SentenceTransformer
```

Imports the `SentenceTransformer` class.

Load Model

```
model = SentenceTransformer(
    "sentence-transformers/all-MiniLM-L6-v2"
)
```

Downloads and loads a pretrained embedding model.

This model converts text into dense vectors.

Input Sentence

```
sentence = "I love Artificial Intelligence"
```

Defines the text we want to embed.

Generate Embedding

```
embedding = model.encode(sentence)
```

Produces a numerical vector representing the sentence.

Print Vector

```
print(embedding)
```

Displays the embedding values.

Print Length

```
print(len(embedding))
```

Shows the embedding dimension (e.g., 384).

4.12 Multiple Sentences Example

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer(
    "sentence-transformers/all-MiniLM-L6-v2"
)

sentences = [
    "I love biryani.",
    "Biryani is my favorite food.",
    "The sky is blue."
```

```
]

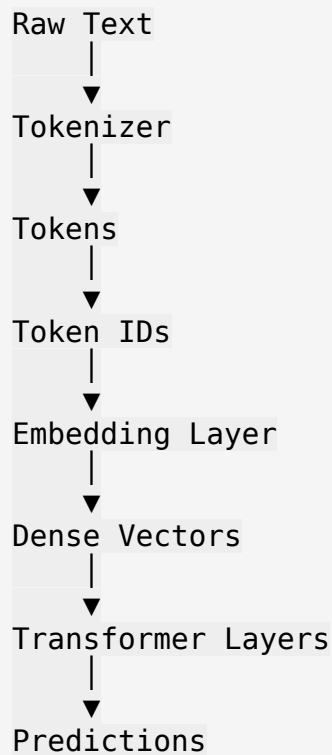
embeddings = model.encode(sentences)

for i, emb in enumerate(embeddings):
    print(f"Sentence {i+1}: {len(emb)} dimensions")
```

Possible Output

```
Sentence 1: 384 dimensions
Sentence 2: 384 dimensions
Sentence 3: 384 dimensions
```

4.13 Embedding Pipeline in an LLM



4.14 Why Are Embeddings Powerful?

Embeddings help models:

- Understand meaning instead of exact words

- □ Measure semantic similarity
- □ Handle synonyms
- □ Generalize to unseen text
- □ Learn relationships between concepts

Example:

"I enjoy coding."

"I like programming."

Although the words differ, their embeddings are likely to be close because they express similar ideas.

4.15 Interview Questions

Q1. What is an embedding?

Answer: An embedding is a dense numerical vector representing the semantic meaning of text.

Q2. Why do LLMs use embeddings?

Answer: Neural networks cannot process raw text. Embeddings convert text into numerical vectors while preserving semantic relationships.

Q3. Are token IDs and embeddings the same?

Answer: No. Token IDs are integer indices, while embeddings are learned floating-point vectors that encode meaning.

Q4. What is the difference between a word embedding and a sentence embedding?

Answer: A word embedding represents a single token or word, whereas a sentence embedding captures the meaning of an entire sentence.

Q5. Can two different sentences have similar embeddings?

Answer: Yes. If they have similar meanings, their embeddings will often be close in vector space.

4.16 Common Mistakes

❑ **Embedding = Token ID**

Wrong. A token ID is just an integer; an embedding is a learned vector.

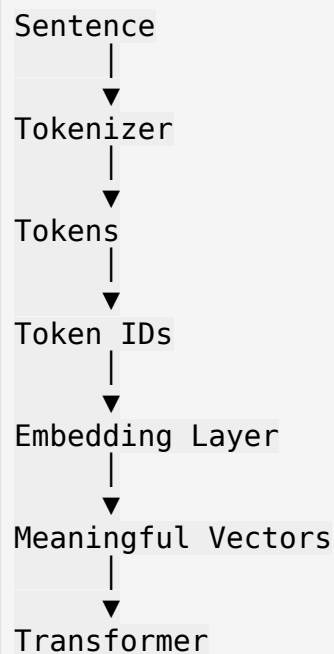
❑ **Embeddings are human-readable**

Wrong. They are numerical representations used internally by the model.

❑ **Every embedding has the same size across all models**

Wrong. Different models use different embedding dimensions (e.g., 384, 768, 1024).

❑ Memory Trick



□ Key Takeaways

- □ **Embeddings convert text into dense numerical vectors.**
 - □ **They preserve semantic meaning**, allowing similar concepts to have nearby representations.
 - □ **Token IDs are not embeddings**; they are only lookup indices.
 - □ **Transformers operate on embeddings**, not raw text.
 - □ **Embedding quality is crucial** for enabling language models to understand and generate meaningful text.
-

□ Interview Tip

If asked:

"What happens after tokenization in an LLM?"

A strong answer is:

"After tokenization, each token ID is mapped to a learned embedding vector. These embeddings capture semantic information and are then processed by the Transformer layers to model relationships and predict subsequent tokens." bold text

```
from sentence_transformers import SentenceTransformer
model=SentenceTransformer(
    "sentence-transformers/all-MiniLM-L6-v2"
)
sentences=[
    "I love biryani.",
    "Biryani is my favorite food.",
    "The sky is blue."
]
embeddings=model.encode(sentences)

for i,emb in enumerate(embeddings):
    print(f"sentence{i+1}: {len(emb)} dimensions")

{"model_id":"838f86d319dd4ca19e6bfb82c2d72c17","version_major":2,"version_minor":0}

sentence1: 384 dimensions
sentence2: 384 dimensions
sentence3: 384 dimensions
```

□ PART 5: Why Embeddings Matter? (Complete Interview Notes)

Goal: Understand why embeddings are one of the most important concepts in LLMs and how they help models understand meaning instead of just matching words.

5.1 Why Do Embeddings Matter?

Imagine these two sentences:

Sentence 1:
"I love programming."

Sentence 2:
"I enjoy coding."

Even though the words are different, **both sentences have nearly the same meaning.**

A human easily understands this.

But how can a computer understand it?

□ **Answer: Embeddings**

Embeddings convert text into vectors where **similar meanings have similar vector representations.**

5.2 Traditional Keyword Search vs Embedding Search

□ **Keyword Search**

Search Query:

Car

Document:

Automobile

Result:

No Match ☐

Because the words are different.

☐ Embedding Search

Car Similar Meaning
Automobile

Result:

Match Found ☐

Embeddings capture **meaning**, not just exact words.

5.3 Semantic Similarity

Look at these sentences:

A: I love biryani.
B: Biryani is my favorite food.
C: The weather is rainy today.

Similarity:

A ↔ B → Very High ☐

A ↔ C → Very Low ☐

Even though A and B use different words, they are semantically similar.

5.4 Real-Life Analogy

Think of a library.

Suppose someone asks:

"I want books about cars."

The librarian also gives books about:

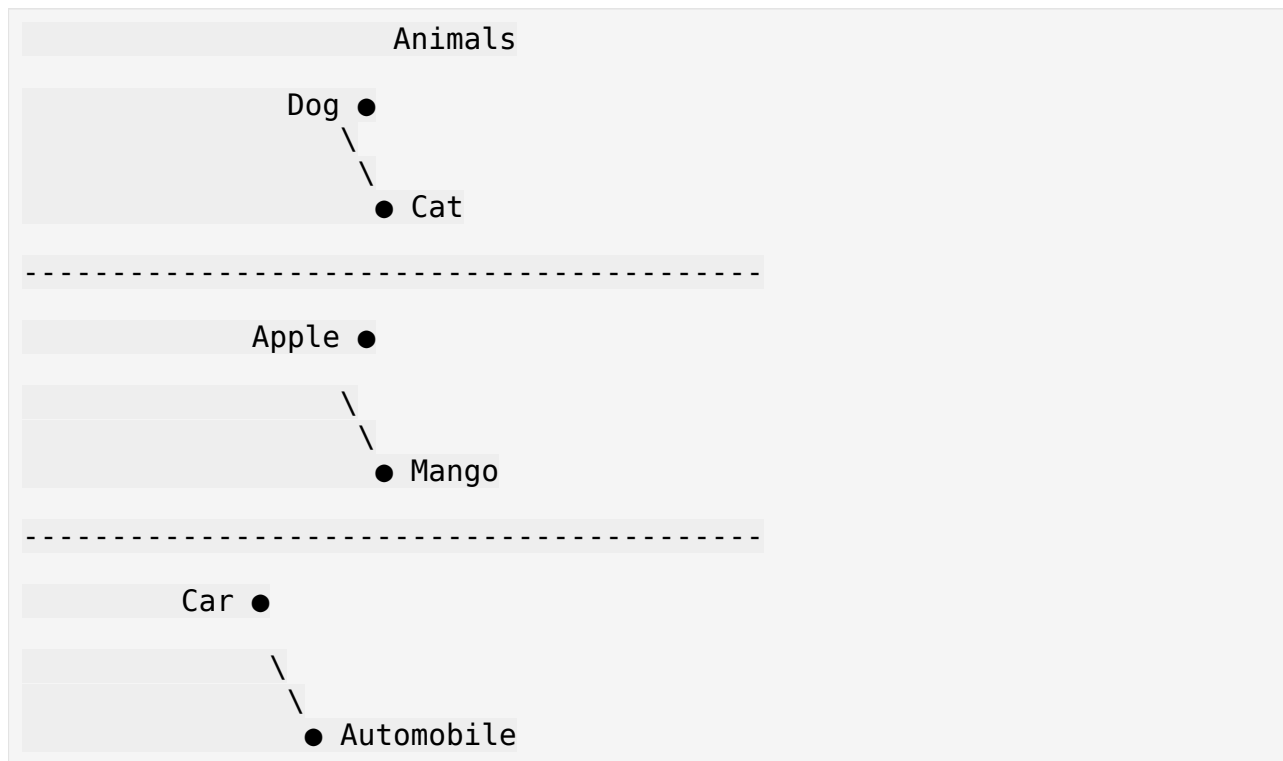
- Automobiles
- Vehicles
- Driving
- Transportation

because they understand the **meaning**.

Embeddings work in a similar way.

5.5 Embedding Space

Imagine every sentence is a point on a map.



Similar concepts are located close together.

Unrelated concepts are far apart.

5.6 Cosine Similarity

The most common way to compare embeddings is **Cosine Similarity**.

It measures the angle between vectors.

$$\text{Similarity} = \cos(\theta)$$

Range:

Score	Meaning
1.0	Exactly similar
0.8	Highly similar
0.5	Moderately related
0.0	Unrelated
-1.0	Opposite direction

5.7 Example

Query:

"I like eating spicy rice."

Documents:

1. Biryani is delicious.
2. Python is a programming language.
3. Cricket is popular in India.

Embedding search returns:

Similarity Scores

Biryani → 0.94

Python → 0.18

Cricket → 0.07

So the system correctly retrieves the biryani-related sentence.

5.8 Python Code Example

```
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity

# Load embedding model
model = SentenceTransformer(
    "sentence-transformers/all-MiniLM-L6-v2"
)

sentences = [
    "I love biryani.",
    "Biryani is my favorite food.",
    "The weather is nice today."
]

# Generate embeddings
embeddings = model.encode(sentences)

# Compare first two sentences
score = cosine_similarity(
    [embeddings[0]],
    [embeddings[1]]
)

print(score)
```

5.9 Expected Output

```
[[0.92]]
```

This means the first and second sentences are highly similar.

(The exact value may vary slightly depending on the model and version.)

5.10 Line-by-Line Code Explanation

Import SentenceTransformer

```
from sentence_transformers import SentenceTransformer
```

Imports the embedding model class.

Import cosine similarity

```
from sklearn.metrics.pairwise import cosine_similarity
```

Imports a function to compare vectors.

Load embedding model

```
model = SentenceTransformer(  
    "sentence-transformers/all-MiniLM-L6-v2"  
)
```

Loads a pretrained model that converts sentences into dense vectors.

Create sentences

```
sentences = [  
    "I love biryani.",  
    "Biryani is my favorite food.",  
    "The weather is nice today."  
)
```

Three input sentences.

Encode

```
embeddings = model.encode(sentences)
```

Generates one embedding vector for each sentence.

Compute similarity

```
score = cosine_similarity(  
    [embeddings[0]],  
    [embeddings[1]]  
)
```

Measures how similar the first and second vectors are.

Print

```
print(score)
```

Displays the similarity score.

5.11 Compare Multiple Sentences

```
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity

model = SentenceTransformer(
    "sentence-transformers/all-MiniLM-L6-v2"
)

texts = [
    "Machine learning is amazing.",
    "Artificial intelligence is fascinating.",
    "I like eating pizza."
]

embeddings = model.encode(texts)

similarity_matrix = cosine_similarity(embeddings)

print(similarity_matrix)
```

5.12 Possible Output

```
[
  [1.00, 0.81, 0.15],
  [0.81, 1.00, 0.18],
  [0.15, 0.18, 1.00]
]
```

Interpretation:

- Sentence 1 and 2 are closely related.
 - Sentence 3 is unrelated to the first two.
-

5.13 Why LLMs Need Embeddings

Without embeddings:

```
Dog != Puppy  
Car != Automobile  
Happy != Joyful
```

With embeddings:

```
Dog ≈ Puppy  
Car ≈ Automobile  
Happy ≈ Joyful
```

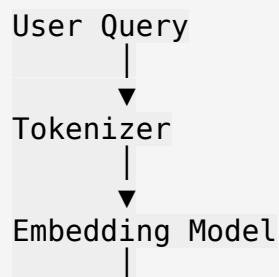
The model understands relationships instead of relying on exact wording.

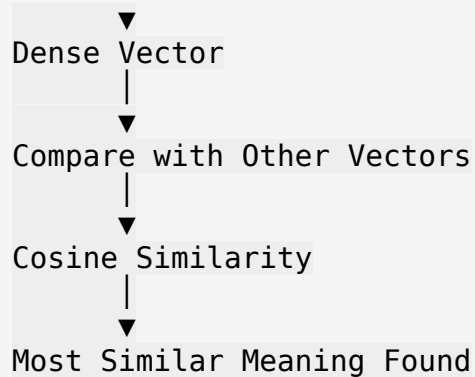
5.14 Applications of Embeddings

Embeddings are used in:

- ☐ Semantic search
 - ☐ Document similarity
 - ☐ Question answering
 - ☐ Text clustering
 - ☐ Recommendation systems
 - ☐ Duplicate detection
 - ☐ Text classification
 - ☐ AI assistants
-

5.15 Workflow





5.16 Common Misconceptions

□ Embeddings are the same as token IDs.

No.

- Token ID → Integer identifier.
 - Embedding → Learned floating-point vector.
-

□ Two identical words always have identical meanings.

Context matters.

Example:

"I sat on the bank."

"The bank approved my loan."

The word **bank** has different meanings depending on context.

Modern contextual embeddings help distinguish these meanings.

□ Bigger vectors always mean better understanding.

Not necessarily.

Quality depends on training, not just vector size.

5.17 Interview Questions

Q1. Why are embeddings important?

Answer: They convert text into dense vectors that preserve semantic meaning, enabling models to compare and understand language mathematically.

Q2. What is semantic similarity?

Answer: It measures how similar two pieces of text are in meaning, even if they use different words.

Q3. What is cosine similarity?

Answer: A metric that measures the angle between embedding vectors to estimate semantic similarity.

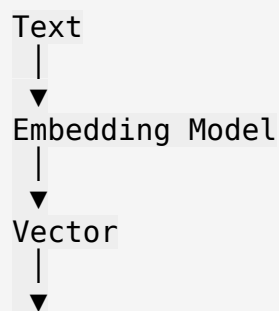
Q4. Can embeddings recognize synonyms?

Answer: Yes. Similar concepts like "car" and "automobile" often produce nearby vectors.

Q5. Why are embeddings better than keyword matching?

Answer: They capture meaning rather than exact word matches, making systems more flexible and accurate.

□ Memory Trick



Cosine Similarity



Meaning-Based Comparison

□ Key Takeaways

- □ Embeddings allow AI to understand **meaning**, not just words.
- □ Similar ideas produce **similar vectors**.
- □ Cosine similarity is commonly used to compare embeddings.
- □ Embeddings power semantic search, recommendations, clustering, and many NLP tasks.
- □ They are a foundational building block of modern LLMs and many AI systems.

□ Interview Tip

If asked:

"Why do embeddings matter in LLMs?"

A strong answer is:

"Embeddings transform text into dense numerical vectors that encode semantic meaning. This allows LLMs to recognize relationships between words and sentences, compare similarity mathematically, and generalize beyond exact keyword matches." bold text

```
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity

# Load embedding model
model = SentenceTransformer(
    "sentence-transformers/all-MiniLM-L6-v2"
)

sentences = [
    "I love biryani.",
    "Biryani is my favorite food.",
    "The weather is nice today."
]

# Generate embeddings
embeddings = model.encode(sentences)

# Compare first two sentences
```

```

score = cosine_similarity(
    [embeddings[0]],
    [embeddings[1]]
)

print(score)

{"model_id": "8c4839364d7f4660844766a9846e9fce", "version_major": 2, "version_minor": 0}

[[0.86562616]]

```

COMPARE MULTIPLE SENTENCES

```

from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity

model = SentenceTransformer(
    "sentence-transformers/all-MiniLM-L6-v2"
)

texts = [
    "Machine learning is amazing.",
    "Artificial intelligence is fascinating.",
    "I like eating pizza."
]

embeddings = model.encode(texts)

similarity_matrix = cosine_similarity(embeddings)

print(similarity_matrix)

{"model_id": "6a8b83c5baed4f31939fda7104e2c109", "version_major": 2, "version_minor": 0}

[[1.0000002  0.6746615  0.10243492]
 [0.6746615  0.9999999  0.15448388]
 [0.10243492 0.15448388 1.0000002 ]]

```

▢ PART 6: Context Window (Complete Interview Notes)

Goal: Understand what a Context Window is, why it is important in LLMs, how it works internally, and how it affects the quality of responses.

6.1 What is a Context Window?

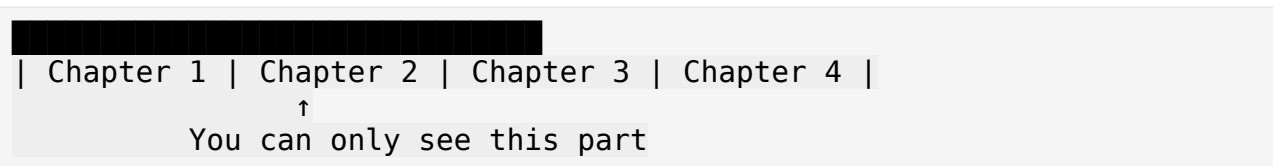
A **Context Window** is the **maximum number of tokens an LLM can read, remember, and process at one time**.

▮ Simple Definition:

A context window is the **memory space** available to the model during a single interaction.

6.2 Real-Life Analogy ▮

Imagine you're reading a book through a small window.



You can only remember and understand the text visible through that window.

Similarly, an LLM only sees the tokens that fit inside its **context window**.

6.3 Example

User Prompt:

```
My name is Yaswanth.  
I study at IIIT Kottayam.  
I like Machine Learning.  
I love Python programming.
```

Suppose the context window can hold **only 10 tokens**.

The model might only retain:

```
I love Python programming.
```

The earlier information may no longer be available for that inference step.

6.4 Context Window in LLM Workflow



6.5 Tokens vs Words

⚠ **Context windows are measured in tokens, not words.**

Example:

Sentence:
"I love Artificial Intelligence."

Possible tokenization:

["I", "love", "Artificial", "Intelligence", "."]

Here:

- Words = 4
- Tokens $\approx$ 5

Another sentence may have:

- 10 words
- 15 or 20 tokens

depending on the tokenizer.

6.6 Example of Token Counting

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("gpt2")

text = "I love learning Large Language Models."

tokens = tokenizer.tokenize(text)
ids = tokenizer.encode(text)

print("Tokens:", tokens)
print("Number of Tokens:", len(ids))
```

6.7 Expected Output

```
Tokens:
['I', 'love', 'learning', 'Large', 'Language', 'Models', '.']

Number of Tokens:
7
```

(Exact tokenization may vary depending on the tokenizer.)

6.8 Line-by-Line Code Explanation

Import tokenizer

```
from transformers import AutoTokenizer
```

Imports Hugging Face's tokenizer.

Load GPT-2 tokenizer

```
tokenizer = AutoTokenizer.from_pretrained("gpt2")
```

Downloads and initializes the tokenizer.

Input text

```
text = "I love learning Large Language Models."
```

Stores the sentence.

Tokenize

```
tokens = tokenizer.tokenize(text)
```

Splits text into model-specific tokens.

Encode

```
ids = tokenizer.encode(text)
```

Converts tokens into integer IDs.

Print count

```
len(ids)
```

Returns the total number of tokens.

6.9 What Happens If the Context Window Is Exceeded?

Suppose:

```
Context Window = 8 Tokens
```

Input:

```
This is a very long sentence that contains many words and exceeds the  
allowed context length.
```

Possible behavior:

Original Input

This is a very long sentence that contains many words...

Only Last Part Used

...contains many words and exceeds the limit.

Some applications instead split the text into chunks before processing.

6.10 Python Demonstration of Truncation

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("gpt2")

text = """
Large Language Models are revolutionizing artificial intelligence by
understanding and generating human language efficiently.
"""

encoded = tokenizer(
    text,
    truncation=True,
    max_length=10
)

print(encoded["input_ids"])
```

6.11 Explanation

```
truncation=True
```

Automatically cuts off tokens beyond the specified limit.

```
max_length=10
```

Restricts the output to 10 tokens.

```
encoded["input_ids"]
```

Displays the token IDs after truncation.

6.12 Why Context Window Matters

A larger context window enables the model to:

- Remember earlier conversation
 - Read longer documents
 - Summarize lengthy articles
 - Analyze large code files
 - Maintain coherence over extended interactions
-

6.13 Small vs Large Context Window

Small Window

```
User:  
My favorite color is blue.
```

```
...  
(after many messages)
```

```
User:  
What's my favorite color?
```

```
Model:  
I don't know. 
```

Large Window

```
User:  
My favorite color is blue.
```

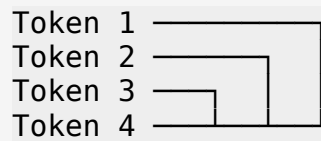
```
...  
(after many messages)
```

```
User:  
What's my favorite color?
```

Model:
Your favorite color is blue. □

6.14 Relationship with Attention

Every token inside the context window can potentially attend to other tokens.



The larger the context, the more relationships the model can consider.

6.15 Context Window vs Memory

Many beginners confuse these concepts.

Context Window	Long-Term Memory
Tokens processed in one inference	Persistent information across sessions
Temporary	Can be stored externally
Exists during generation	Not inherent to the base model

A context window is **not** permanent memory.

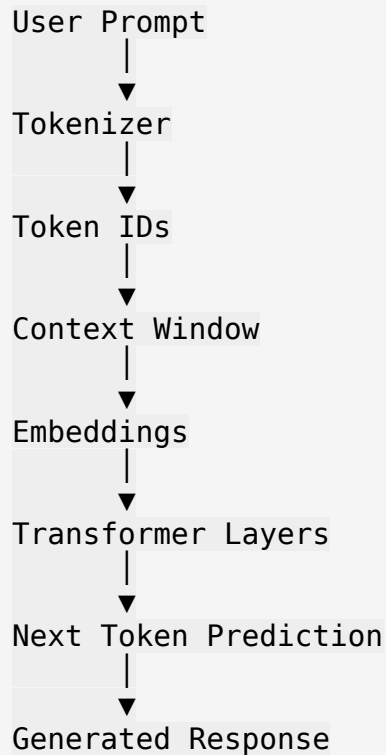
6.16 Common Sizes

Different models support different limits.

Example Model	Typical Context Size
GPT-2	~1K tokens
GPT-3 era	Several thousand tokens
Modern LLMs	Tens or even hundreds of thousands of tokens

(The exact limit depends on the specific model version.)

6.17 Complete Pipeline



6.18 Interview Questions

Q1. What is a context window?

Answer: A context window is the maximum number of tokens an LLM can process simultaneously during inference.

Q2. Is it measured in words?

Answer: No. It is measured in **tokens**, and one word may correspond to multiple tokens.

Q3. What happens when the limit is exceeded?

Answer: The input may be truncated or split into smaller chunks, depending on the application.

Q4. Why is a larger context window useful?

Answer: It enables the model to consider more information at once, improving long-document understanding and conversational continuity.

Q5. Does the context window represent permanent memory?

Answer: No. It is temporary and only applies to the tokens processed during a particular inference.

6.19 Common Mistakes

❑ **Context window = number of words**

Wrong. It is based on **tokens**.

❑ **The model remembers everything forever**

Wrong. It only processes information within its available context (unless an external memory system is used).

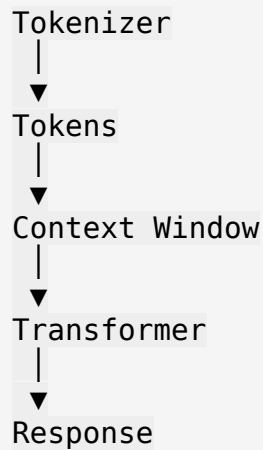
❑ **Longer prompts always produce better answers**

Wrong. Exceeding the context limit may lead to truncation or loss of earlier information.

❑ Memory Trick

Text





Think of the **context window** as the model's temporary working memory for the current request.

□ Key Takeaways

- □ A **context window** is the maximum number of **tokens** an LLM can process at one time.
 - □ It determines how much information the model can use while generating a response.
 - □ It is measured in **tokens**, not words or characters.
 - □ Inputs longer than the limit may be **truncated** or **chunked**.
 - □ Larger context windows improve long-document reasoning and conversational continuity.
-

□ Interview Tip

If asked:

"What is a context window in an LLM?"

A strong answer is:

"A context window is the maximum number of input and generated tokens that a language model can consider during a single inference. It acts as the model's temporary working memory and directly affects how much prior information can influence its response."

bold text

Example of Token Counting


```

from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("gpt2")

text = "I love learning Large Language Models."

tokens = tokenizer.tokenize(text)
ids = tokenizer.encode(text)

print("Tokens:", tokens)
print("Number of Tokens:", len(ids))

{"model_id": "09338e69f5b641e0a2445acc5d807364", "version_major": 2, "version_minor": 0}

{"model_id": "f981251e94a74cc9b436cf3166ee2777", "version_major": 2, "version_minor": 0}

{"model_id": "dbda65c19c4f4873818e21055eb498f0", "version_major": 2, "version_minor": 0}

{"model_id": "b5e89237cbbd4e108a723793ed6990fa", "version_major": 2, "version_minor": 0}

{"model_id": "5a91f78c0a404ebbb2115e4e1f2af3f3", "version_major": 2, "version_minor": 0}

Tokens: ['I', 'love', 'learning', 'Large', 'Language', 'Models', '.']
Number of Tokens: 7

```

Python Demonstration of Truncation

```

from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("gpt2")

text = """
Large Language Models are revolutionizing artificial intelligence by
understanding and generating human language efficiently.
"""

encoded = tokenizer(
    text,
    truncation=True,
    max_length=10
)

print(encoded["input_ids"])

[198, 21968, 15417, 32329, 389, 5854, 2890, 11666, 4430, 416]

```

□ PART 7: Transformer Architecture (Master Notes for LLM Interviews)

Goal: Understand the complete Transformer architecture—the foundation of modern LLMs like GPT, Llama, Gemini, Claude, and Qwen.

7.1 What is a Transformer?

A **Transformer** is a deep learning architecture introduced in the 2017 paper "**Attention Is All You Need**".

It processes **all input tokens in parallel** using **self-attention**, making it much faster and more effective than RNNs and LSTMs.

In Simple Words

Transformer = A neural network architecture that understands relationships between all words in a sentence simultaneously.

7.2 Why Were Transformers Introduced?

□ Problem with RNN

I → Love → Learning → AI

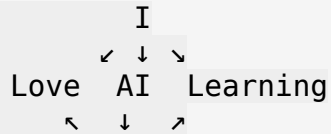
- Processes one word at a time
 - Slow
 - Difficult to remember long contexts
-

□ Problem with LSTM

LSTM improved memory but still:

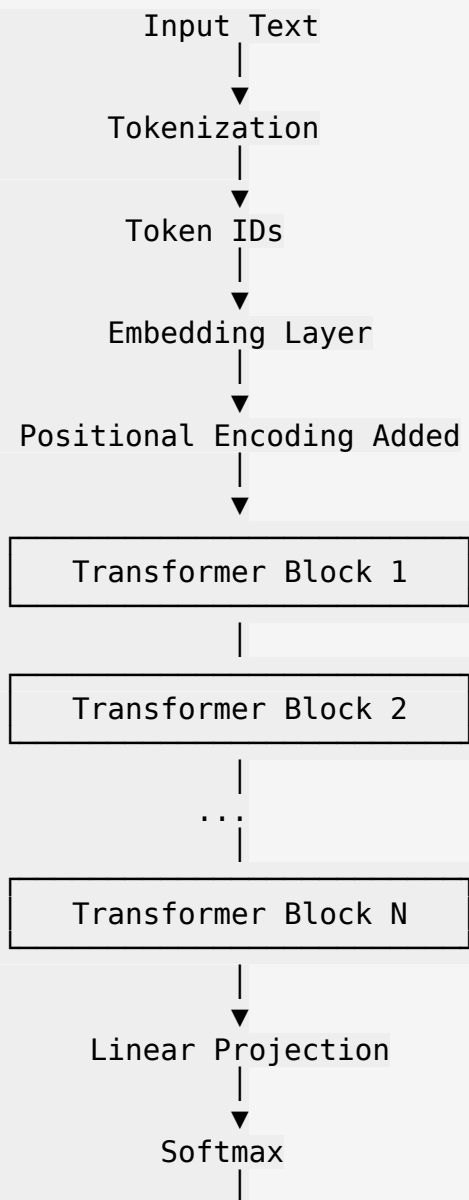
- Sequential processing
 - Hard to parallelize
 - Expensive for very long sequences
-

□ Transformer Solution



Every token can interact with every other token **at the same time**.

7.3 Overall Transformer Pipeline



7.4 Components of a Transformer

A Transformer mainly consists of:

- [] Tokenizer
 - [] Embedding Layer
 - [] Positional Encoding
 - [] Self-Attention
 - [] Multi-Head Attention
 - [] Feed Forward Network (FFN)
 - [] Residual Connections
 - [] Layer Normalization
 - [] Linear Layer
 - [] Softmax
-

7.5 Step 1 – Tokenization

Input:

I love AI

↓

["I", "love", "AI"]

↓

[40, 1842, 9552]

These token IDs become the input to the model.

7.6 Step 2 – Embedding Layer

Each token ID is mapped to a dense vector.

Example:

40

↓

[0.12, 0.91, -0.44, ...]

Every token becomes a high-dimensional representation.

7.7 Step 3 – Positional Encoding

Why?

Transformers process tokens **in parallel**.

Without extra information:

Dog bites man

Man bites dog

could appear identical.

So positional information is added.

Example

Token	Position
-------	----------

Dog	1
-----	---

bites	2
-------	---

man	3
-----	---

The embedding is updated:

Final Input =

Token Embedding

+

Position Embedding

This allows the model to understand word order.

7.8 Step 4 – Self-Attention □□□□□

This is the **heart of the Transformer**.

Sentence:

Virat Kohli scored a century.

He celebrated.

Question:

Who is **He**?

Self-attention learns:

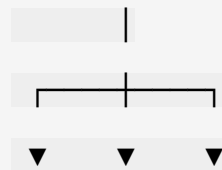
He → Virat Kohli

instead of incorrectly connecting to "century".

7.9 Query (Q), Key (K), Value (V)

Every token creates three vectors:

Embedding



Query Key Value

Think of them as:

Component	Purpose
Query	What am I looking for?
Key	What information do I have?
Value	What information should I provide?

Example

Sentence:

The cat sat on the mat.

Token:

cat

Produces:

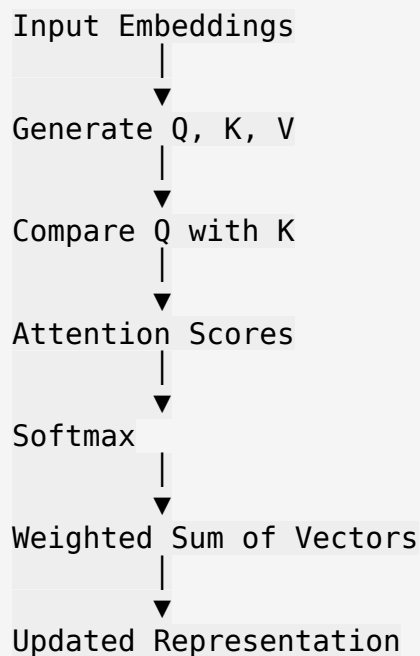
Query = [0.3, 0.7]

Key = [0.8, 0.2]

Value = [0.1, 0.9]

The Query compares with all Keys to determine which Values are most relevant.

7.10 Self-Attention Workflow



7.11 Multi-Head Attention

Instead of using one attention mechanism:

Sentence

↓

Attention Head 1

Transformers use many:

Sentence

└─→ Head 1

└─→ Head 2

└─→ Head 3

└─→ Head 4

Different heads may focus on:

- Grammar
- Subject-object relationships
- Pronouns
- Long-distance dependencies

The outputs are combined.

7.12 Feed Forward Network (FFN)

After attention, each token passes through a neural network.

Attention Output

↓

Linear Layer

↓

Activation (e.g., GELU)

↓

Linear Layer

↓

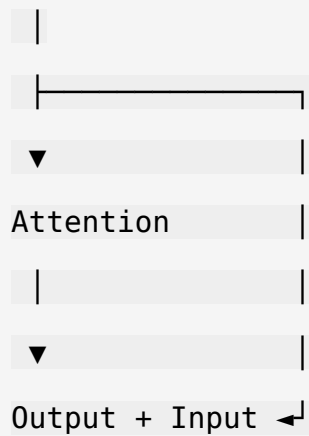
Updated Features

This increases the model's expressive power.

7.13 Residual Connections

Instead of discarding the original input:

Input



Benefits:

- Easier optimization
 - Better gradient flow
 - Stable deep networks
-

7.14 Layer Normalization

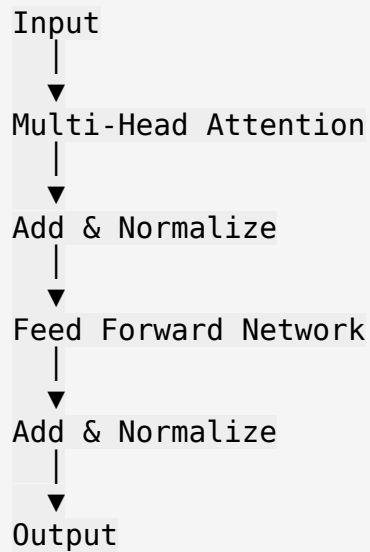
Normalizes activations after computations.

Benefits:

- Stable training
 - Faster convergence
 - Reduced numerical issues
-

7.15 Transformer Block

A single Transformer block looks like:



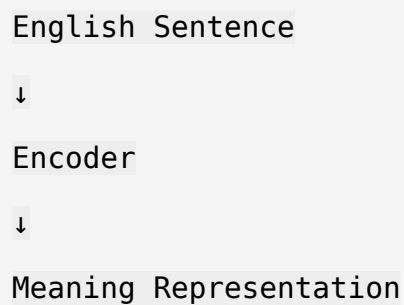
Modern LLMs stack dozens or even hundreds of these blocks.

7.16 Encoder vs Decoder

Encoder

Reads and understands input.

Example:



Used heavily in BERT-style models.

Decoder

Generates text one token at a time.

Prompt

↓

Decoder

↓

Next Token

↓

Next Token

↓

Sentence

GPT models primarily use decoder-only architectures.

Encoder–Decoder

Input

↓

Encoder

↓

Context

↓

Decoder

↓

Generated Output

Used by models like T5 for tasks such as translation and summarization.

7.17 Python Demo: Inspect Tokenization Before the Transformer

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("gpt2")

text = "Transformers changed AI."

inputs = tokenizer(text)

print(inputs)
```

Possible Output

```
{
  'input_ids': [41762, 3194, 9552, 13],
  'attention_mask': [1, 1, 1, 1]
}
```

`input_ids` are then converted into embeddings before entering the Transformer.

7.18 Line-by-Line Explanation

```
from transformers import AutoTokenizer
```

Imports the tokenizer.

```
tokenizer = AutoTokenizer.from_pretrained("gpt2")
```

Loads GPT-2's tokenizer.

```
inputs = tokenizer(text)
```

Tokenizes the text and prepares model-ready inputs.

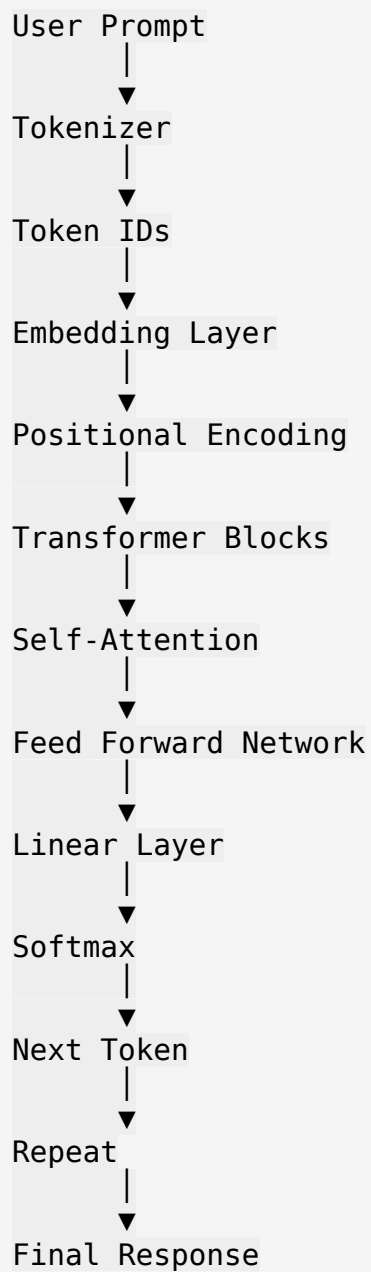
```
inputs["input_ids"]
```

Returns integer IDs for the tokens.

`attention_mask`

Marks valid tokens (1) and padding (0) for models that require padding.

7.19 Complete LLM Pipeline



7.20 Interview Questions

Q1. What is a Transformer?

A Transformer is a neural network architecture that processes sequences using self-attention instead of recurrence, enabling efficient parallel computation.

Q2. Why are Transformers better than RNNs?

- Parallel processing
 - Better long-range dependency modeling
 - Faster training
 - Better scalability
-

Q3. What is Self-Attention?

A mechanism that allows each token to determine which other tokens are most relevant when forming its representation.

Q4. What are Query, Key, and Value?

They are learned projections used by the attention mechanism:

- Query: what the token is looking for
 - Key: what information each token offers
 - Value: the information carried by each token
-

Q5. Why is Positional Encoding needed?

Because Transformers process tokens in parallel and therefore require explicit information about token order.

Q6. What is Multi-Head Attention?

Multiple attention mechanisms running in parallel so the model can capture different types of relationships simultaneously.

❑ Common Mistakes

"Transformers read words one by one."

Wrong.

They process all input tokens in parallel during encoding.

"Q, K, and V are the original words."

Wrong.

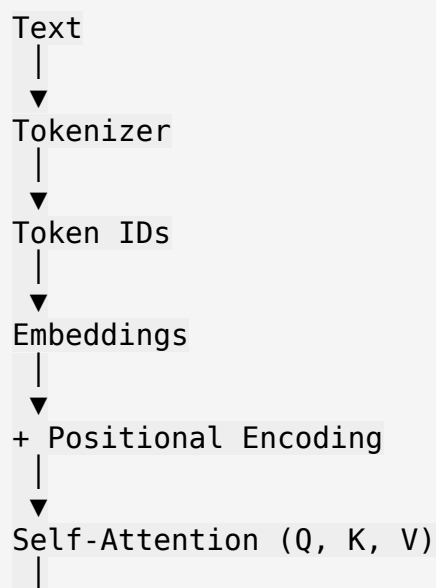
They are learned vector projections derived from embeddings.

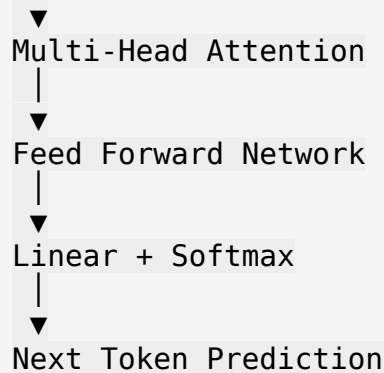
"Attention and Self-Attention are identical."

Not always.

Self-attention operates within the same sequence, while attention more generally can operate across different sequences.

❑ Memory Trick





□ Key Takeaways

- □ The **Transformer** is the foundation of modern LLMs.
- □ It replaces recurrence with **self-attention**.
- □ **Embeddings + positional information** are fed into Transformer blocks.
- □ **Q, K, and V** drive the attention mechanism.
- □ **Multi-head attention** captures different relationships in parallel.
- □ **Residual connections** and **layer normalization** stabilize training.
- □ GPT-style LLMs use this architecture repeatedly to generate one token after another.

□ Interview Tip (Strong Answer)

"A Transformer first tokenizes text, converts tokens into embeddings, adds positional information, computes self-attention using Query, Key, and Value vectors, refines representations through feed-forward networks and normalization, and finally predicts the next token using a probability distribution. Modern LLMs stack many such Transformer blocks to model complex language patterns."

This level of explanation is typically sufficient for **AI Engineer and LLM interview discussions** on Transformer architecture. **bold text**

□ PART 8: Attention Mechanism (Complete Interview Notes)

Goal: Master the Attention Mechanism—the core innovation that made Transformers and modern LLMs like GPT, Llama, Gemini, and Claude possible.

8.1 What is Attention?

The **Attention Mechanism** allows a model to **focus on the most relevant words or tokens** when understanding or generating text.

Simple Definition

Attention tells the model which words are important and which words should receive more focus.

8.2 Real-Life Analogy ☐

Imagine you are reading this sentence:

"Rohit gave the ball to Virat because he was injured."

When you read **"he"**, your brain automatically tries to determine **who "he" refers to**.

Humans naturally pay attention to the relevant words.

LLMs do exactly the same thing using the **Attention Mechanism**.

8.3 Why Do We Need Attention?

Consider:

The trophy doesn't fit in the suitcase because it is too large.

Question:

What is **"it"**?

Possible references:

- Trophy ☐
- Suitcase ☐

Without attention, the model might get confused.

With attention:

it
|

▼
[Trophy]

The model learns that **"it"** refers to **"trophy"**.

8.4 Before Attention (RNN Problem)

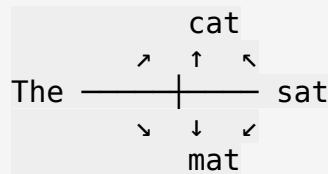
Word1 → Word2 → Word3 → Word4 → Word5

Problems:

- Information from earlier words may fade.
 - Long-distance dependencies are difficult.
 - Slow sequential computation.
-

8.5 With Attention

Every word can look at every other word.



This enables richer understanding of relationships.

8.6 Example Sentence

The cat sat on the mat because it was tired.

When processing **"it"**, attention assigns high importance to:

it → cat

rather than unrelated words like **"the"** or **"on"**.

8.7 Self-Attention

Self-Attention means:

Every token attends to other tokens within the same input sequence.

Example:

Sentence:

Alice loves programming because she enjoys solving problems.

When processing:

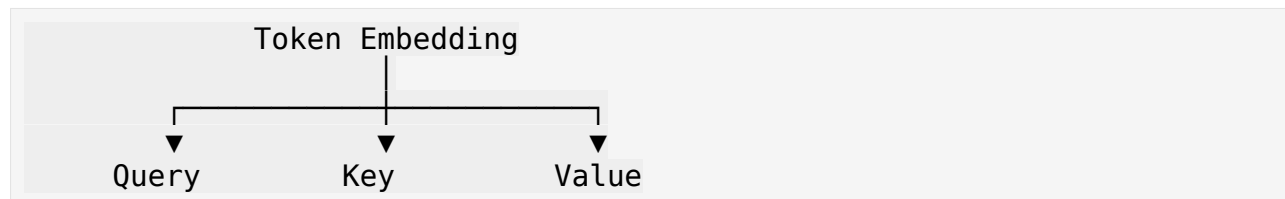
she

the model focuses on:

she → Alice

8.8 Query (Q), Key (K), and Value (V)

Every token generates three vectors:



Intuition

Vector	Meaning
Query	What am I searching for?
Key	What information do I offer?
Value	What information do I contain?

8.9 Restaurant Analogy

Imagine entering a restaurant.

You say:

"I want biryani."

This is your **Query**.

The menu contains all dishes.

These are the **Keys**.

The waiter brings the selected food.

That is the **Value**.

Customer Request



Query

Menu Items



Keys

Selected Dish



Value

8.10 Attention Score

The Query is compared with every Key.

Example:

Sentence:

The cat chased the mouse.

When processing:

cat

Scores:

cat → 0.95

mouse → 0.60

the → 0.10

chased → 0.75

Higher score = more attention.

8.11 Softmax

Raw scores:

[3.2, 1.8, 0.4]

Softmax converts them into probabilities:

[0.73, 0.21, 0.06]

Properties:

- All values are between 0 and 1.
 - They sum to 1.
-

8.12 Weighted Sum

Each Value vector is multiplied by its attention weight.

Attention Weight × Value

↓

Weighted Values

↓

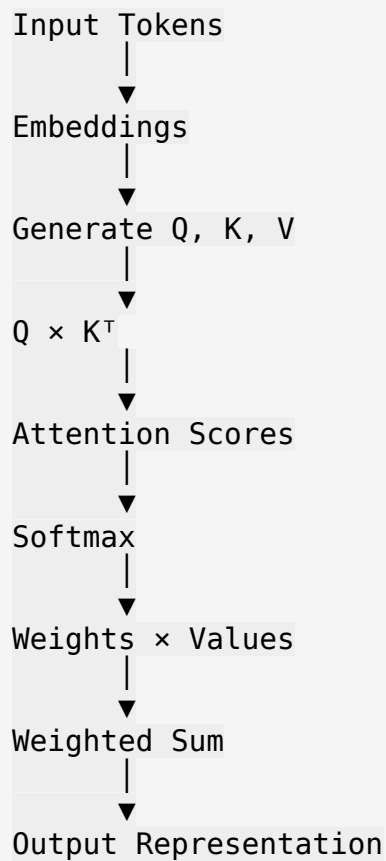
Sum

↓

Final Representation

This becomes the updated representation of the token.

8.13 Complete Self-Attention Workflow



8.14 Multi-Head Attention

Instead of computing attention once:

Sentence

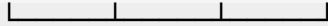
↓

Head 1

Transformers compute multiple attention heads simultaneously:



Head1 Head2 Head3 Head4



Combine Outputs

Each head can specialize:

- Grammar
 - Pronouns
 - Subject-object relations
 - Long-range dependencies
-

8.15 Python Demo: Viewing Attention (Conceptual)

```
from transformers import AutoTokenizer, AutoModel

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
model = AutoModel.from_pretrained(
    "bert-base-uncased",
    output_attentions=True
)

text = "The cat sat on the mat."
inputs = tokenizer(text, return_tensors="pt")
outputs = model(**inputs)
attentions = outputs.attentions
print(len(attentions))
```

8.16 Possible Output

12

Meaning:

BERT Base contains **12 Transformer layers**, each producing attention matrices.

8.17 Line-by-Line Explanation

Import tokenizer

```
from transformers import AutoTokenizer
```

Loads the tokenizer utility.

Import model

```
from transformers import AutoModel
```

Loads the neural network architecture.

Enable attention outputs

```
output_attentions=True
```

Requests the model to return attention weights.

Tokenize

```
inputs = tokenizer(  
    text,  
    return_tensors="pt"  
)
```

Converts text into PyTorch tensors.

Run model

```
outputs = model(**inputs)
```

Feeds the tokens into BERT.

Access attention matrices

```
outputs.attentions
```


Returns attention scores for each Transformer layer.

8.18 Visual Example

Sentence:

The boy kicked the ball because it was dirty.

When processing:

it

Attention may focus on:

boy	→ 0.15
kicked	→ 0.10
ball	→ 0.70 □
dirty	→ 0.05

The model correctly links **"it"** to **"ball"**.

8.19 Why Attention Changed AI

Before attention:

- Long-distance relationships were difficult.
- Sequential computation slowed training.

After attention:

- Better understanding of context.
 - Parallel processing.
 - Superior performance.
 - Foundation of all modern LLMs.
-

8.20 Attention vs Self-Attention

Attention	Self-Attention
Can compare different sequences	Operates within the same sequence
Used in encoder-decoder interactions	Used inside Transformer blocks
General concept	Special case of attention

8.21 Common Misconceptions

□ Attention is memory.

No.

Attention is a mechanism for dynamically weighting relationships between tokens.

□ Highest attention always means most important meaning.

Not necessarily.

Attention weights provide useful information but are not a perfect explanation of model reasoning.

□ Self-attention only looks at nearby words.

Wrong.

It can connect tokens that are far apart in the sequence.

8.22 Interview Questions

Q1. What is the Attention Mechanism?

Answer: It enables the model to focus on the most relevant tokens when computing representations for a given token.

Q2. What is Self-Attention?

Answer: A mechanism where each token attends to other tokens in the same input sequence to capture contextual relationships.

Q3. What are Query, Key, and Value?

Answer:

- **Query (Q):** Represents what a token is looking for.
 - **Key (K):** Represents what information each token provides.
 - **Value (V):** Contains the information passed forward after attention weighting.
-

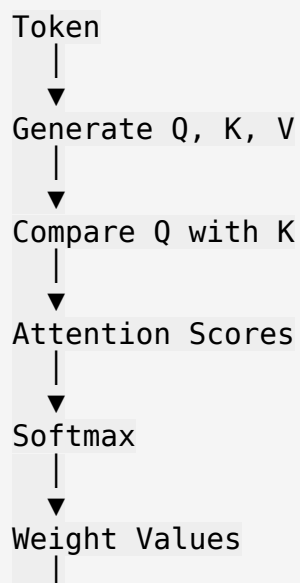
Q4. Why is Multi-Head Attention useful?

Answer: Different heads can capture different linguistic and semantic relationships simultaneously.

Q5. What role does Softmax play?

Answer: Softmax converts raw attention scores into normalized probabilities that sum to one, allowing the model to weight Value vectors appropriately.

□ Memory Trick



□ Key Takeaways

- □ **Attention** helps the model identify the most relevant words.
 - □ **Self-Attention** lets every token interact with every other token in the same sequence.
 - □ **Query, Key, and Value** are the core components used to compute attention.
 - □ **Softmax** transforms attention scores into probabilities.
 - □ **Multi-Head Attention** allows the model to learn multiple kinds of relationships in parallel.
 - □ The Attention Mechanism is the **core innovation behind the Transformer architecture and modern LLMs**.
-

□ Interview Tip (High-Scoring Answer)

“The Attention Mechanism enables a Transformer to dynamically determine which tokens are most relevant when processing a given token. Each token is projected into Query, Key, and Value vectors. Queries are compared with Keys to compute attention scores, Softmax normalizes those scores into weights, and the weighted Values are combined to produce context-aware representations. Multi-Head Attention extends this idea by allowing multiple attention patterns to be learned simultaneously.”

This explanation is strong enough for **AI Engineer, Generative AI, and LLM-focused interviews**.
bold text

□ PART 9: Training Phase of LLMs (Complete Interview Notes)

Goal: Understand how a Large Language Model (LLM) is trained from scratch, including data collection, tokenization, embeddings, forward pass, loss calculation, backpropagation, optimizer updates, and epochs.

9.1 What is the Training Phase?

The **Training Phase** is the process where an LLM **learns patterns from massive amounts of text data** by repeatedly predicting the next token and updating its parameters based on errors.

Simple Definition

Training = Teaching an LLM by showing it billions of text examples and allowing it to improve through repeated prediction and correction.

9.2 Real-Life Analogy ☐

Imagine a student learning English.

Teacher:

"The capital of France is ____"

Student answers:

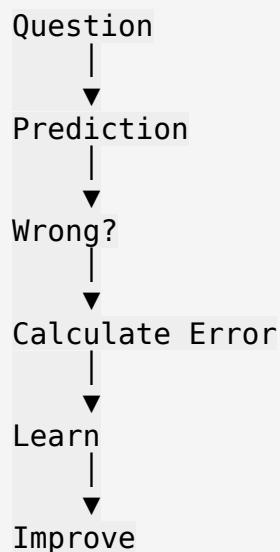
"London" ☐

Teacher corrects:

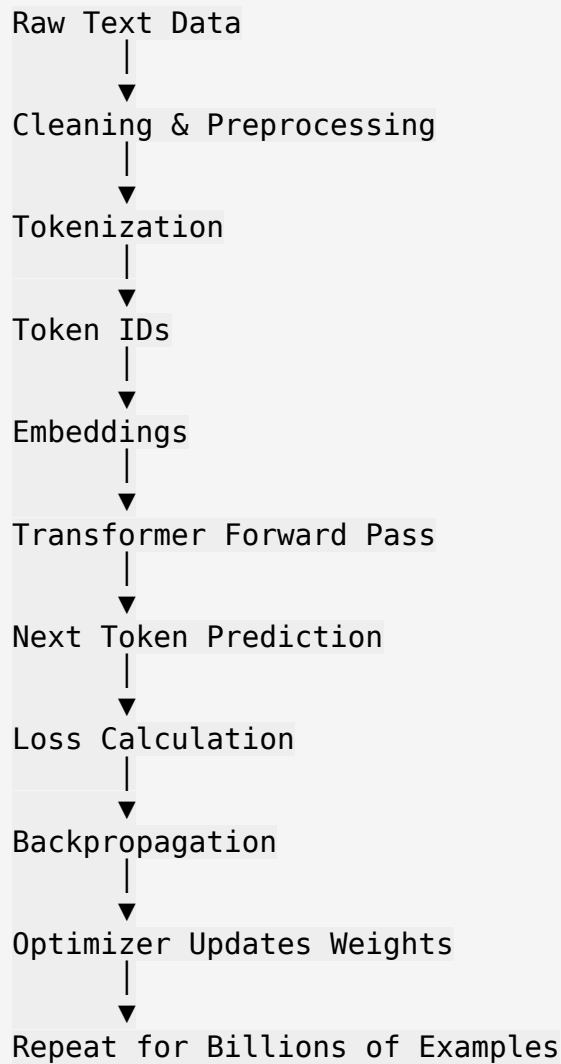
"Paris" ☐

The student learns from the mistake.

LLMs learn the same way.



9.3 Complete Training Pipeline



9.4 Step 1 – Collect Data

Training data may include:

- Books
- Web pages
- Articles
- Source code
- Research papers
- Documentation
- Question-answer pairs

Example:

```
I love Machine Learning.  
AI is transforming healthcare.  
Python is a popular programming language.
```

9.5 Step 2 – Tokenization

Raw text:

```
I love AI
```

↓

Tokens:

```
["I", "love", "AI"]
```

↓

Token IDs:

```
[40, 1842, 9552]
```

The Transformer processes numbers—not raw text.

9.6 Step 3 – Embeddings

Token IDs:

```
[40, 1842, 9552]
```

↓

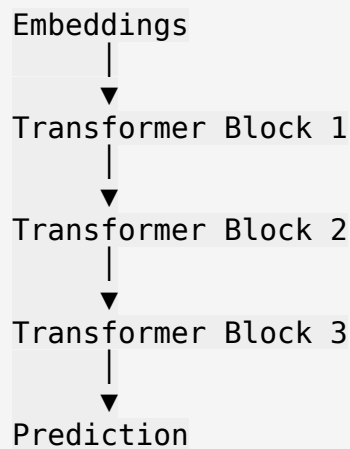
Embedding vectors:

```
[  
  [0.21, -0.13, 0.72],  
  [0.88,  0.42, 0.19],  
  [0.55, -0.66, 0.44]  
]
```

Embeddings encode semantic information.

9.7 Step 4 – Forward Pass

The embeddings are passed through Transformer layers.



This process is called the **forward pass**.

9.8 Step 5 – Next Token Prediction

Training sentence:

I love machine

Target:

learning

The model predicts:

Prediction: education □
Correct: learning □

9.9 Step 6 – Loss Calculation

The model measures how wrong it was.

Example:

Correct Answer:
learning

Model Prediction:
education

Loss:

High Error

If prediction is accurate:

Prediction:
learning

Loss:
Very Small

9.10 What is Loss?

Loss is a numerical value representing prediction error.

Good Prediction



Low Loss (0.1)

Bad Prediction



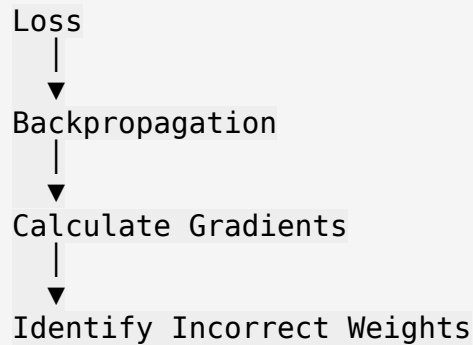
High Loss (4.8)

The objective is:

Minimize Loss

9.11 Step 7 – Backpropagation

After computing loss:



Backpropagation determines how each parameter contributed to the error.

9.12 Step 8 – Optimizer Updates Weights

Common optimizers:

- SGD
- Adam
- AdamW

The optimizer adjusts model weights.



After many updates, predictions improve.

9.13 Step 9 – Repeat Millions of Times

Training loop:

Example 1

↓

Learn

↓

Example 2

↓

Learn

↓

Example 3

↓

Learn

↓

...

↓

Billions of Examples

9.14 Epochs

An **epoch** means processing the entire training dataset once.

Example:

Dataset:
1000 samples

1 Epoch =
1000 processed once

3 epochs:

1000

↓

1000

↓

1000

The model sees the dataset three times.

9.15 Python Example: Tiny Training Concept

```
from transformers import AutoTokenizer  
  
tokenizer = AutoTokenizer.from_pretrained("gpt2")  
  
text = "I love learning AI."  
  
tokens = tokenizer.tokenize(text)  
  
ids = tokenizer.encode(text)  
  
print(tokens)  
print(ids)
```

Possible Output

```
['I', 'love', 'learning', 'AI', '.']  
[40, 1842, 4673, 9552, 13]
```

These IDs are what the model would use during training.

9.16 Line-by-Line Explanation

Import tokenizer

```
from transformers import AutoTokenizer
```

Imports Hugging Face's tokenizer.

Load tokenizer

```
tokenizer = AutoTokenizer.from_pretrained("gpt2")
```

Loads GPT-2's vocabulary and tokenization rules.

Input text

```
text = "I love learning AI."
```

Defines one training example.

Tokenize

```
tokens = tokenizer.tokenize(text)
```

Splits the sentence into tokens.

Encode

```
ids = tokenizer.encode(text)
```

Maps tokens to integer IDs.

Print

```
print(ids)
```

Displays the numerical representation used by the model.

9.17 Conceptual Training Loop (Pseudo-code)

```
for batch in training_data:
    # Convert text to tokens
    inputs = tokenizer(batch)

    # Model prediction
    outputs = model(inputs)
```

```
# Compare prediction with target
loss = compute_loss(outputs)

# Compute gradients
loss.backward()

# Update weights
optimizer.step()

# Clear old gradients
optimizer.zero_grad()
```

9.18 Explanation of the Training Loop

`tokenizer(batch)`

Converts raw text into model inputs.

`model(inputs)`

Runs a forward pass through the Transformer.

`compute_loss(outputs)`

Calculates prediction error.

`loss.backward()`

Performs backpropagation and computes gradients.

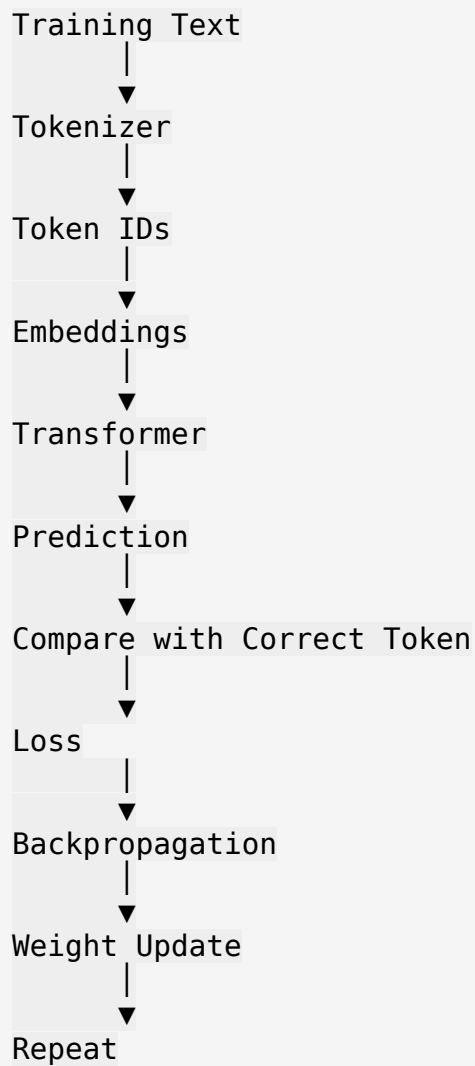
`optimizer.step()`

Updates parameters to reduce future errors.

`optimizer.zero_grad()`

Resets gradients before processing the next batch.

9.19 Visual Training Flow



9.20 Example

Training sentence:

The sky is blue.

Input:

The sky is

Target:

blue

Model predicts:

green □

Loss is high.

After many updates:

blue □

Loss decreases.

9.21 Key Terms

Term	Meaning
Forward Pass	Model generates predictions
Loss	Measures prediction error
Backpropagation	Computes gradients
Optimizer	Updates parameters
Epoch	One complete pass through the dataset
Batch	Small subset processed at once
Gradient	Indicates how weights should change

9.22 Interview Questions

Q1. What is the training phase of an LLM?

Answer: The training phase is the process of learning from large text datasets by predicting tokens, computing loss, and updating model parameters through backpropagation and optimization.

Q2. What is a forward pass?

Answer: A forward pass is when inputs move through the neural network to produce predictions.

Q3. What is loss?

Answer: Loss is a numerical measure of how different the model's prediction is from the correct answer.

Q4. What is backpropagation?

Answer: Backpropagation computes gradients that indicate how parameters should be adjusted to reduce loss.

Q5. What does an optimizer do?

Answer: An optimizer updates model weights using computed gradients to improve future predictions.

Q6. What is an epoch?

Answer: One complete pass through the entire training dataset.

❑ Common Mistakes

"Training means memorizing data."

Wrong.

Training adjusts parameters to learn statistical patterns rather than simply storing text.

"Loss should increase."

Wrong.

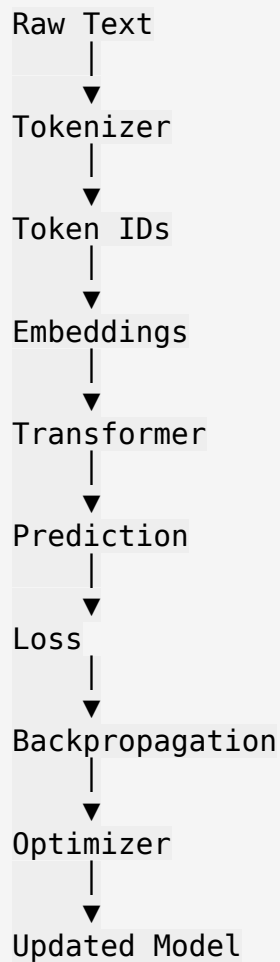
The objective is to **decrease loss over time**.

"Backpropagation makes predictions."

Wrong.

The forward pass makes predictions; backpropagation computes gradients for learning.

□ Memory Trick



□ Key Takeaways

- □ Training teaches an LLM by repeatedly predicting the next token.
- □ The pipeline is: **Text → Tokens → Embeddings → Transformer → Prediction → Loss → Backpropagation → Weight Update.**
- □ **Loss** measures prediction error.

- **Backpropagation** computes gradients.
 - **Optimizers** adjust weights to improve performance.
 - Repeating this process over massive datasets enables LLMs to learn language patterns.
-

□ Interview Tip (Strong Answer)

If asked:

"How is an LLM trained?"

You can answer:

"An LLM is trained by tokenizing large text corpora, converting tokens into embeddings, processing them through Transformer layers to predict the next token, computing a loss against the correct token, and then using backpropagation and an optimizer such as AdamW to update the model parameters. This process is repeated across billions of examples over many epochs, allowing the model to learn linguistic and semantic patterns."

This is an **AI Engineer-level explanation** and is suitable for interviews. **bold text**

□ PART 10: Inference Phase in LLMs (Complete Interview Notes)

Goal: Understand what happens **after an LLM is trained**. Learn how models like ChatGPT generate responses step by step during inference.

10.1 What is Inference?

After training is complete, the model enters the **Inference Phase**.

Inference is the process of using a trained LLM to generate predictions or responses for new user inputs.

Simple Definition

Inference = Using a trained model to answer questions or generate text without updating its weights.

10.2 Training vs Inference

Training	Inference
Learns from data	Uses learned knowledge
Updates weights	Weights remain fixed
Computes loss	No loss computation
Uses backpropagation	No backpropagation
Takes days/weeks	Usually takes milliseconds or seconds

10.3 Real-Life Analogy ☐

Imagine a student.

During Training

Teacher:

$2 + 3 = ?$

Student:

4 ☐

Teacher:

Wrong, it's 5.

Student learns.

During Inference

Teacher:

$2 + 3 = ?$

Student:

5 ☐

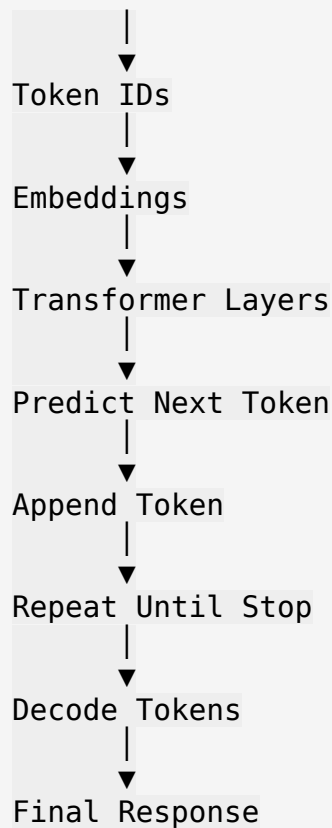
No learning occurs during inference—the student simply uses existing knowledge.

10.4 Complete Inference Pipeline

User Prompt



Tokenizer



10.5 Example

User enters:

I love learning

The model predicts:

I love learning AI

Then:

I love learning AI because

Then:

I love learning AI because it

Then:

I love learning AI because it helps people.

The model generates **one token at a time**.

10.6 Step 1 – User Prompt

Example:

Explain Machine Learning.

This is raw text.

10.7 Step 2 – Tokenization

Explain Machine Learning.

↓

["Explain", "Machine", "Learning", "."]

↓

Token IDs:

[1823, 8532, 6521, 13]

10.8 Step 3 – Embeddings

Token IDs become vectors.

1823

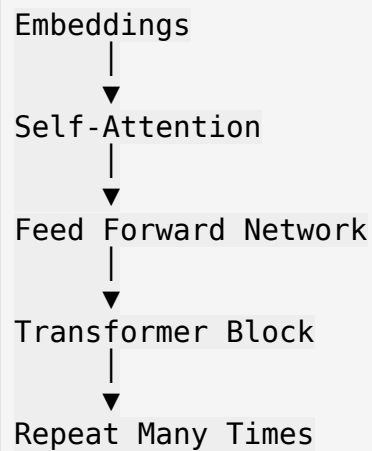
↓

[0.34, -0.18, 0.92, ...]

The Transformer operates on these vectors.

10.9 Step 4 – Transformer Processing

The embeddings pass through many Transformer blocks.



Each layer refines the token representations.

10.10 Step 5 – Next Token Prediction

Suppose the prompt is:

The capital of France is

The model computes probabilities:

Token	Probability
Paris	0.96
London	0.02
Rome	0.01
Berlin	0.01

The highest-probability token is selected (or sampled, depending on decoding strategy).

Output:

Paris

10.11 Step 6 – Append the Token

The generated token is appended to the prompt.

Before:

The capital of France is

↓

After:

The capital of France is Paris

Now the updated sequence is fed back into the model.

10.12 Step 7 – Repeat

The process repeats:

Prompt

↓

Predict Token

↓

Append Token

↓

Predict Again

↓

Append Again

↓

Continue Until Finished

This is called **autoregressive generation**.

10.13 Step 8 – Stopping Conditions

Generation stops when:

- The model emits an **end-of-sequence token (<EOS>)**
 - The maximum token limit is reached
 - Another application-defined stopping criterion is met
-

10.14 Python Example: Text Generation

```
from transformers import pipeline

generator = pipeline(
    "text-generation",
    model="distilgpt2"
)

prompt = "Artificial Intelligence is"

result = generator(
    prompt,
    max_new_tokens=20
)

print(result[0]["generated_text"])
```

10.15 Possible Output

Artificial Intelligence is transforming healthcare, education, finance, and many other industries.

(The exact wording may vary because generation is probabilistic.)

10.16 Line-by-Line Code Explanation

Import

```
from transformers import pipeline
```

Imports Hugging Face's pipeline API.

Create generator

```
generator = pipeline(  
    "text-generation",  
    model="distilgpt2"  
)
```

Loads a pretrained GPT-style language model.

Prompt

```
prompt = "Artificial Intelligence is"
```

Defines the input text.

Generate

```
generator(  
    prompt,  
    max_new_tokens=20  
)
```

Generates up to 20 additional tokens.

Print

```
print(result[0]["generated_text"])
```

Displays the final generated response.

10.17 Internally, the Model Predicts One Token at a Time

Prompt:

```
I love
```

Step 1:

I love learning

Step 2:

I love learning about

Step 3:

I love learning about AI

Step 4:

I love learning about AI every day.

This sequential generation continues until a stopping condition is reached.

10.18 Probability Distribution

The model does **not** directly output a word.

Instead, it predicts probabilities.

Example:

Prompt:
I drink

Probabilities:

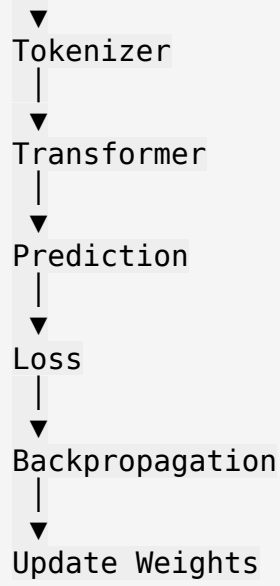
water	→ 0.65
coffee	→ 0.20
tea	→ 0.10
juice	→ 0.05

One token is then selected based on the decoding strategy.

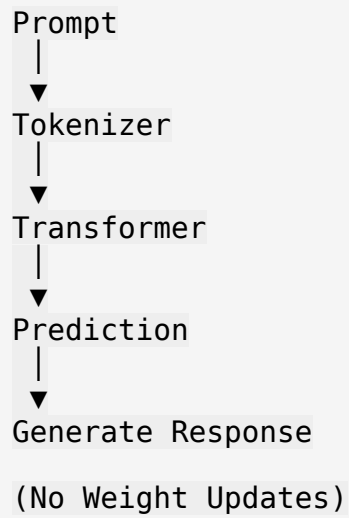
10.19 Inference vs Training Diagram

TRAINING

Text
|



INFERENCE



10.20 Mini Example

User:

Python is

Possible generated tokens:

Python is

↓

a

↓

popular

↓

programming

↓

language.

Final output:

Python is a popular programming language.

10.21 Key Terms

Term	Meaning
Prompt	User input text
Inference	Using a trained model to generate output
Autoregressive	Generating one token at a time
Logits	Raw prediction scores before normalization
Softmax	Converts logits into probabilities
Decoding	Choosing the next token from probabilities

10.22 Interview Questions

Q1. What is inference in an LLM?

Answer: Inference is the process of using a trained language model to generate predictions or responses without modifying its learned parameters.

Q2. Does inference update model weights?

Answer: No. During inference, the model performs only forward computation and does not update its parameters.

Q3. How does an LLM generate text?

Answer: It tokenizes the prompt, converts tokens into embeddings, processes them through Transformer layers, predicts the next token, appends it to the sequence, and repeats until a stopping condition is reached.

Q4. What is autoregressive generation?

Answer: Generating text one token at a time, where each newly generated token becomes part of the input for predicting the next token.

Q5. What is the role of Softmax during inference?

Answer: Softmax converts raw model outputs (logits) into a probability distribution over possible next tokens.

❑ Common Mistakes

"The model generates an entire sentence at once."

Wrong.

It generates **one token at a time**.

"Inference performs learning."

Wrong.

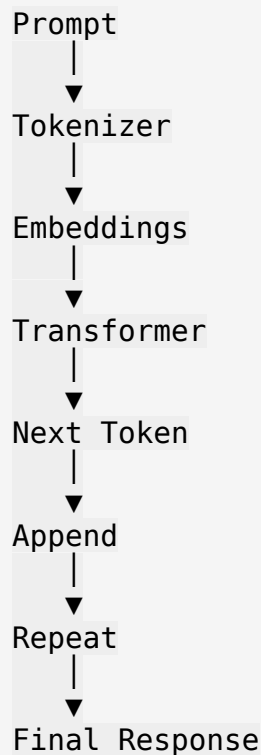
Learning occurs only during training.

"The model always picks the highest-probability token."

Not necessarily.

Depending on decoding settings (e.g., temperature, top-k, top-p), it may sample a lower-probability token.

□ Memory Trick



□ Key Takeaways

- □ **Inference** is the process of using a trained LLM to generate responses.
 - □ **No weights are updated** during inference.
 - □ The model predicts **one token at a time** in an autoregressive manner.
 - □ The pipeline is: **Prompt → Tokenization → Embeddings → Transformer → Probability Distribution → Token Selection → Repeat → Final Response**
 - □ Training teaches the model; inference applies what it has learned.
-

□ Interview Tip (Strong Answer)

If asked:

"Explain the inference phase of an LLM."

A high-quality answer is:

"During inference, the user's prompt is tokenized and converted into embeddings. These embeddings pass through the Transformer network, which computes a probability distribution over the next possible token. A decoding strategy selects one token, appends it to the existing sequence, and repeats the process until an end condition is met. Unlike training, inference does not compute loss or update model weights."

This explanation is appropriate for **AI Engineer, Machine Learning Engineer, and LLM-focused technical interviews. bold text**

□ PART 11: Temperature & Creativity in LLMs (Complete Interview Notes)

Goal: Understand how an LLM decides whether to produce **safe and predictable answers** or **creative and diverse responses** using **Temperature, Top-k, and Top-p sampling**.

11.1 What is Temperature?

Temperature is a hyperparameter that controls the **randomness** of token selection during text generation.

Simple Definition

Low Temperature = More deterministic and focused outputs **High Temperature = More creative and diverse outputs**

Temperature **does not change the model's knowledge**. It only changes **how the next token is selected**.

11.2 Real-Life Analogy □

Imagine you are choosing a restaurant.

Low Temperature (Very Safe)

Top Rated Restaurant ☐☐☐☐

Probability:
95%

You almost always choose it.

High Temperature (Creative)

Restaurant A → 40%
Restaurant B → 30%
Restaurant C → 20%
Restaurant D → 10%

You might try something new.

11.3 Example Prompt

Prompt:

Once upon a time

Temperature = 0.0

Output:

Once upon a time there was a king who ruled a peaceful kingdom.

Very predictable.

Temperature = 0.7

Output:

Once upon a time a curious inventor built a flying bicycle.

Balanced creativity.

Temperature = 1.5

Output:

Once upon a time glowing dragons played chess on floating islands made of clouds.

Highly creative and sometimes less coherent.

11.4 Probability Distribution

Suppose the model predicts:

Token	Probability
Paris	0.80
London	0.10
Rome	0.06
Berlin	0.04

Low Temperature

Paris □

Almost always selected.

High Temperature

Paris
London
Rome
Berlin

All become more likely to be sampled.

11.5 Visual Representation

Low Temperature



High Temperature



Distribution becomes flatter.

11.6 Temperature Values

Temperature	Behavior
0.0	Almost deterministic
0.2	Very focused
0.5	Slight creativity
0.7	Balanced
1.0	Default-like randomness
1.5+	Very creative, may become unstable

11.7 Python Example

```
from transformers import pipeline

generator = pipeline(
    "text-generation",
```

```
    model="distilgpt2"
)
prompt = "The future of AI is"
result = generator(
    prompt,
    max_new_tokens=20,
    temperature=0.2
)
print(result[0]["generated_text"])
```

11.8 Possible Output

The future of AI is expected to improve healthcare, education, and scientific research.

With:

```
temperature=1.2
```

Possible output:

The future of AI is filled with imaginative robotic ecosystems and unexpected digital civilizations.

11.9 Line-by-Line Explanation

Import

```
from transformers import pipeline
```

Imports Hugging Face's pipeline utility.

Load model

```
generator = pipeline(
    "text-generation",
```

```
    model="distilgpt2"  
)
```

Creates a GPT-based text generator.

Prompt

```
prompt = "The future of AI is"
```

Defines the input text.

Temperature

```
temperature=0.2
```

Reduces randomness and encourages the highest-probability tokens.

Generate

```
max_new_tokens=20
```

Limits generation to 20 additional tokens.

11.10 What is Top-k Sampling?

Instead of considering **every token** in the vocabulary, the model keeps only the **top k most likely tokens**.

Example:

Vocabulary:

Paris
London
Rome
Berlin
Tokyo
Delhi
Sydney

If:

```
top_k = 3
```

Only:

```
Paris  
London  
Rome
```

are eligible for selection.

11.11 Python Example: Top-k

```
result = generator(  
    "AI will",  
    max_new_tokens=20,  
    top_k=5  
)
```

Meaning:

Only the **5 highest-probability tokens** are considered at each generation step.

11.12 What is Top-p (Nucleus Sampling)?

Instead of fixing the number of candidate tokens, Top-p keeps selecting tokens until their **cumulative probability** reaches p .

Example:

Token	Probability
Paris	0.60
London	0.20
Rome	0.10
Berlin	0.05
Tokyo	0.05

If:

```
top_p = 0.9
```

Keep:

Paris
London
Rome

because:

$$0.60 + 0.20 + 0.10 = 0.90$$

11.13 Python Example: Top-p

```
result = generator(  
    "The scientist discovered",  
    max_new_tokens=20,  
    top_p=0.9  
)
```

This dynamically chooses candidate tokens whose cumulative probability reaches 90%.

11.14 Greedy Decoding

Greedy decoding always chooses the highest-probability token.

Example:

Prompt:
The capital of France is

Probabilities:

Paris → 0.95
London → 0.03
Rome → 0.02

Greedy output:

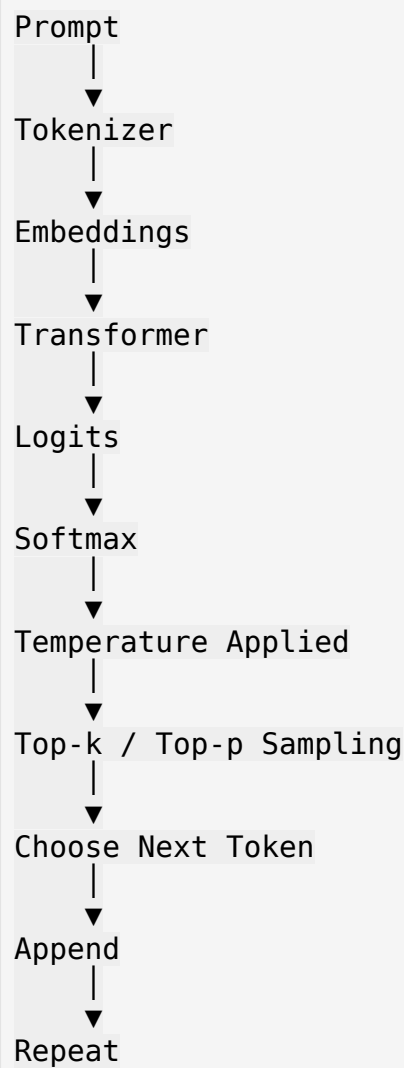
Paris

Always.

11.15 Comparison

Method	Behavior
Greedy	Picks highest-probability token
Temperature	Adjusts randomness
Top-k	Limits to k best candidates
Top-p	Limits by cumulative probability

11.16 Complete Generation Pipeline



11.17 Practical Uses

Low Temperature

Best for:

- Mathematical calculations
 - Coding
 - Medical facts
 - Legal summaries
 - Technical documentation
-

High Temperature

Best for:

- Story writing
 - Poetry
 - Brainstorming
 - Creative marketing
 - Fiction
-

11.18 Interview Questions

Q1. What is temperature in an LLM?

Answer: Temperature controls the randomness of token selection during text generation. Lower values produce more deterministic outputs, while higher values encourage diversity and creativity.

Q2. Does temperature change the model's knowledge?

Answer: No. It only changes how the next token is selected from the predicted probability distribution.

Q3. What is Top-k sampling?

Answer: Top-k sampling restricts candidate tokens to the k highest-probability options before sampling.

Q4. What is Top-p sampling?

Answer: Top-p (nucleus sampling) selects the smallest set of tokens whose cumulative probability reaches a threshold p , then samples from that set.

Q5. When should temperature be low?

Answer: When accuracy and consistency are more important than creativity, such as coding, factual Q&A, or technical writing.

❑ Common Mistakes

"Temperature makes the model smarter."

❑ Wrong.

It only changes **how predictions are sampled**, not what the model knows.

"Temperature = 0 guarantees perfect answers."

❑ Wrong.

It simply makes the output more deterministic.

"High temperature always means better creativity."

❑ Wrong.

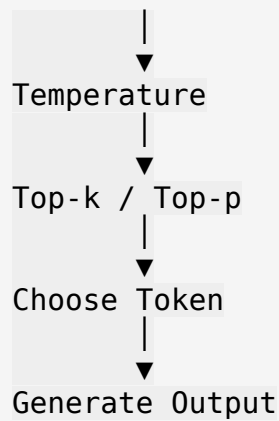
Very high values can produce incoherent or nonsensical text.

❑ Memory Trick

Model Prediction



Probability Distribution



□ Key Takeaways

- □ **Temperature controls randomness, not knowledge.**
- □ **Low temperature → predictable, factual responses.**
- □ **High temperature → creative, diverse responses.**
- □ **Top-k limits the number of candidate tokens.**
- □ **Top-p limits candidates based on cumulative probability.**
- □ These decoding strategies strongly influence how an LLM generates text.

□ Interview Tip (High-Scoring Answer)

If asked:

“What is temperature in an LLM?”

A strong answer is:

“Temperature is a decoding parameter that adjusts the randomness of next-token selection. Lower temperatures make the output more deterministic by favoring high-probability tokens, while higher temperatures flatten the probability distribution and encourage more diverse and creative generations. It affects sampling behavior but does not change the model’s learned knowledge or parameters.”

This explanation is appropriate for **LLM, AI Engineer, and Generative AI** interviews. **bold text**

▯ PART 12: How ChatGPT Actually Answers a Question (Complete Interview Notes)

Goal: Understand the **complete end-to-end pipeline** of how ChatGPT and other LLMs generate an answer—from the moment you type a prompt until the final response appears.

12.1 What Happens When You Ask ChatGPT a Question?

Suppose you type:

What is Artificial Intelligence?

Does ChatGPT search Google?

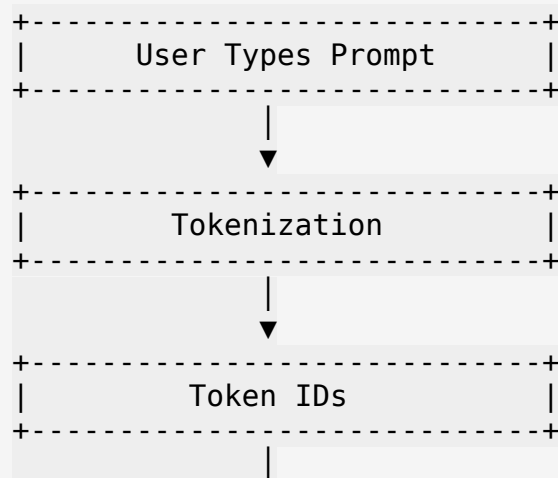
▯ **No.**

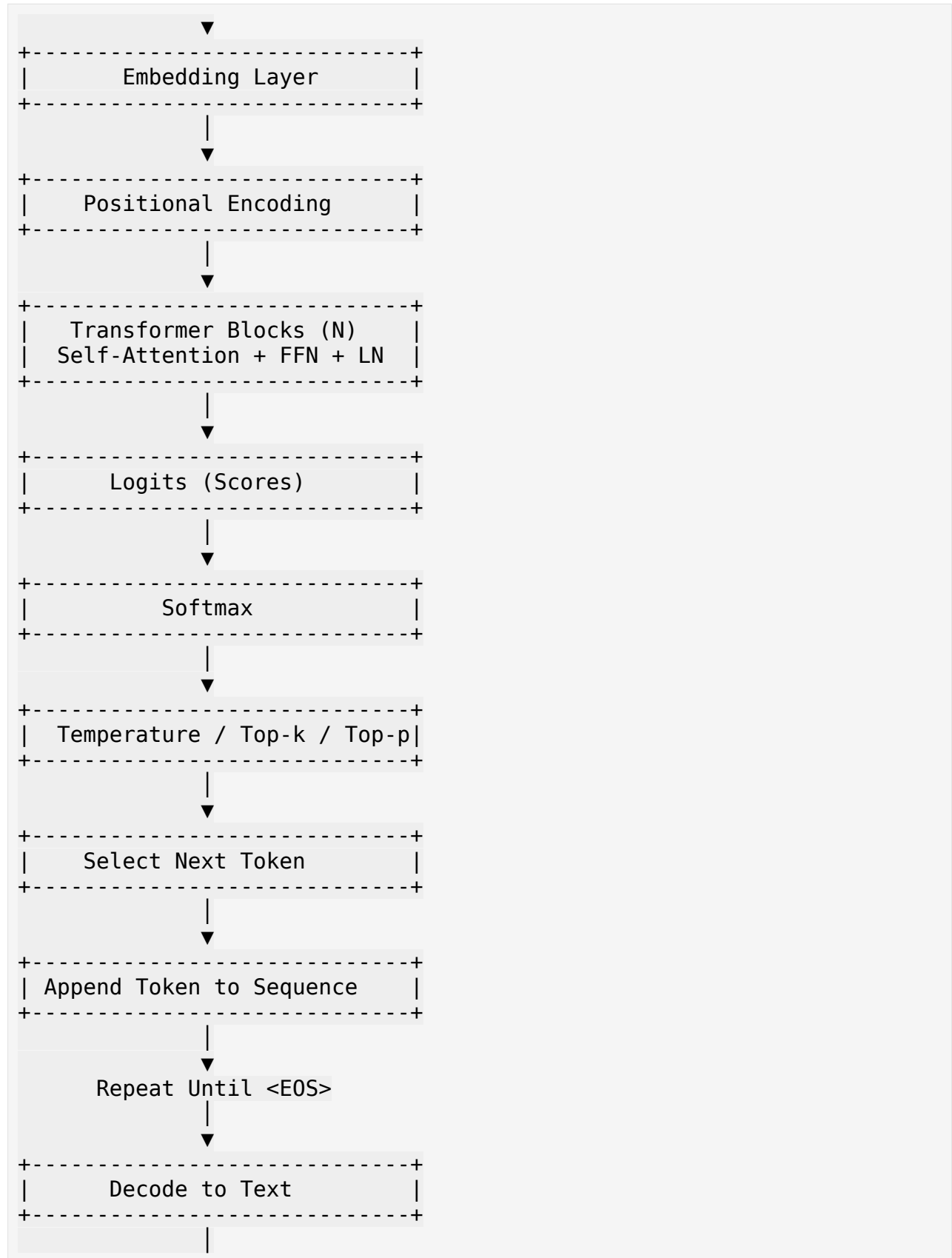
Does ChatGPT have a human writing the answer?

▯ **No.**

Instead, it uses a trained **Transformer-based Large Language Model (LLM)** to **predict the next token repeatedly** until the answer is complete.

12.2 End-to-End Pipeline





▼

```
+-----+
| Final Response |
+-----+
```

12.3 Step 1 – User Prompt

Example:

```
Who invented Python?
```

This is plain text entered by the user.

12.4 Step 2 – Tokenization

The tokenizer splits the sentence.

```
Who invented Python?
```

↓

```
["Who", "invented", "Python", "?"]
```

Then converts tokens into IDs.

```
[8241, 12345, 9876, 30]
```

The exact IDs depend on the tokenizer.

12.5 Step 3 – Embeddings

Each token ID becomes a dense vector.

```
8241
```

↓

```
[0.32, -0.41, 0.91, ...]
```

Now the model has mathematical representations instead of text.

12.6 Step 4 – Positional Encoding

The Transformer needs to know token order.

Example:

Dog bites man

≠

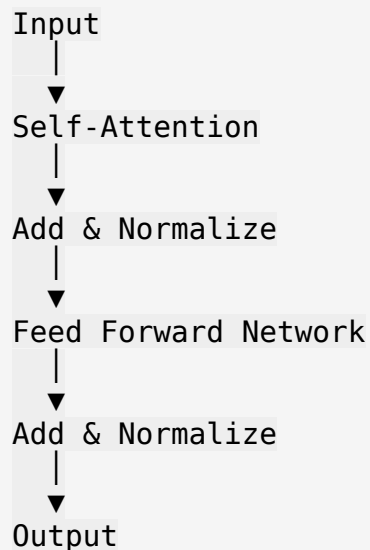
Man bites dog

Positional information is added to embeddings so word order is preserved.

12.7 Step 5 – Transformer Processing

The embeddings pass through multiple Transformer blocks.

Each block performs:



Modern LLMs repeat this process many times.

12.8 Step 6 – Self-Attention Understands Context

Example:

```
Guido created Python because he wanted simplicity.
```

When processing:

```
he
```

Attention focuses on:

```
he → Guido
```

This helps the model understand references and context.

12.9 Step 7 – Logits

After the Transformer finishes processing, it produces **logits**.

Example:

Prompt:

```
The capital of France is
```

Logits:

```
Paris    → 18.4  
London   →  2.1  
Rome     →  0.9  
Berlin   →  0.7
```

Logits are **raw scores**, not probabilities.

12.10 Step 8 – Softmax

Softmax converts logits into probabilities.

Example:

Token	Probability
Paris	0.97
London	0.02
Rome	0.005
Berlin	0.005

These probabilities sum to **1.0**.

12.11 Step 9 – Temperature & Sampling

The model now chooses the next token.

Possible methods:

- Greedy Decoding
- Temperature Sampling
- Top-k Sampling
- Top-p Sampling

For factual questions:

The capital of France is

↓

Paris

12.12 Step 10 – Append the Token

After selecting:

Paris

the sequence becomes:

The capital of France is Paris

Now this updated sequence is fed back into the model.

12.13 Step 11 – Repeat Again

The model predicts another token.

The capital of France is Paris

↓

.

Then another.

Eventually:

The capital of France is Paris.

Generation stops.

12.14 Autoregressive Generation

ChatGPT generates text like this:

Prompt

↓

Predict Token 1

↓

Append

↓

Predict Token 2

↓

Append

↓

Predict Token 3

↓

Append

↓

Continue

↓

Stop

This is called **autoregressive decoding**.

12.15 Python Example

```
from transformers import pipeline

generator = pipeline(
    "text-generation",
    model="distilgpt2"
)

prompt = "Python was created by"

result = generator(
    prompt,
    max_new_tokens=20,
    temperature=0.7
)

print(result[0]["generated_text"])
```

12.16 Possible Output

Python was created by Guido van Rossum and first released in the early 1990s.

(The wording may vary depending on the model and decoding parameters.)

12.17 Line-by-Line Code Explanation

Import

```
from transformers import pipeline
```

Imports Hugging Face's pipeline API.

Load Model

```
generator = pipeline(  
    "text-generation",  
    model="distilgpt2"  
)
```

Loads a pretrained GPT-style language model.

Prompt

```
prompt = "Python was created by"
```

Provides the initial context for generation.

Generate

```
generator(  
    prompt,  
    max_new_tokens=20,  
    temperature=0.7  
)
```

The model predicts up to 20 additional tokens using moderate randomness.

Print

```
print(result[0]["generated_text"])
```

Displays the generated response.

12.18 Complete Example

User asks:

What is Machine Learning?

Step 1

Tokenizer:

["What", "is", "Machine", "Learning", "?"]

Step 2

Convert to IDs:

[2061, 318, 4572, 8932, 30]

Step 3

Convert to embeddings.

Step 4

Run through Transformer blocks.

Step 5

Predict:

Machine

then

Learning

then

is

then

a

then

field

Eventually producing:

"Machine Learning is a branch of Artificial Intelligence that enables systems to learn patterns from data and make predictions without being explicitly programmed."

12.19 Why Doesn't ChatGPT Generate the Whole Answer at Once?

Because LLMs are trained for **next-token prediction**.

Example:

Prompt:

I love

↓

Predict:
learning

↓

Predict:
about

↓

Predict:
Artificial

↓

Predict:
Intelligence

One token at a time.

12.20 Does ChatGPT Think Like Humans?

□ No.

It performs:

- Matrix multiplications
- Attention computations
- Probability estimation
- Token prediction

It does **not** possess consciousness or emotions.

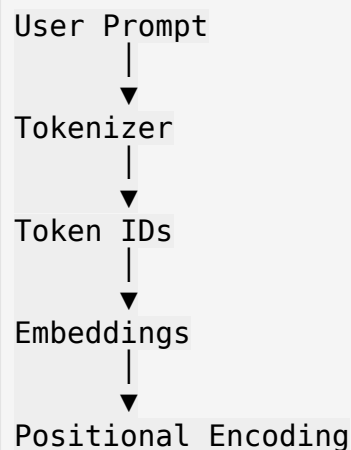
12.21 Can ChatGPT Make Mistakes?

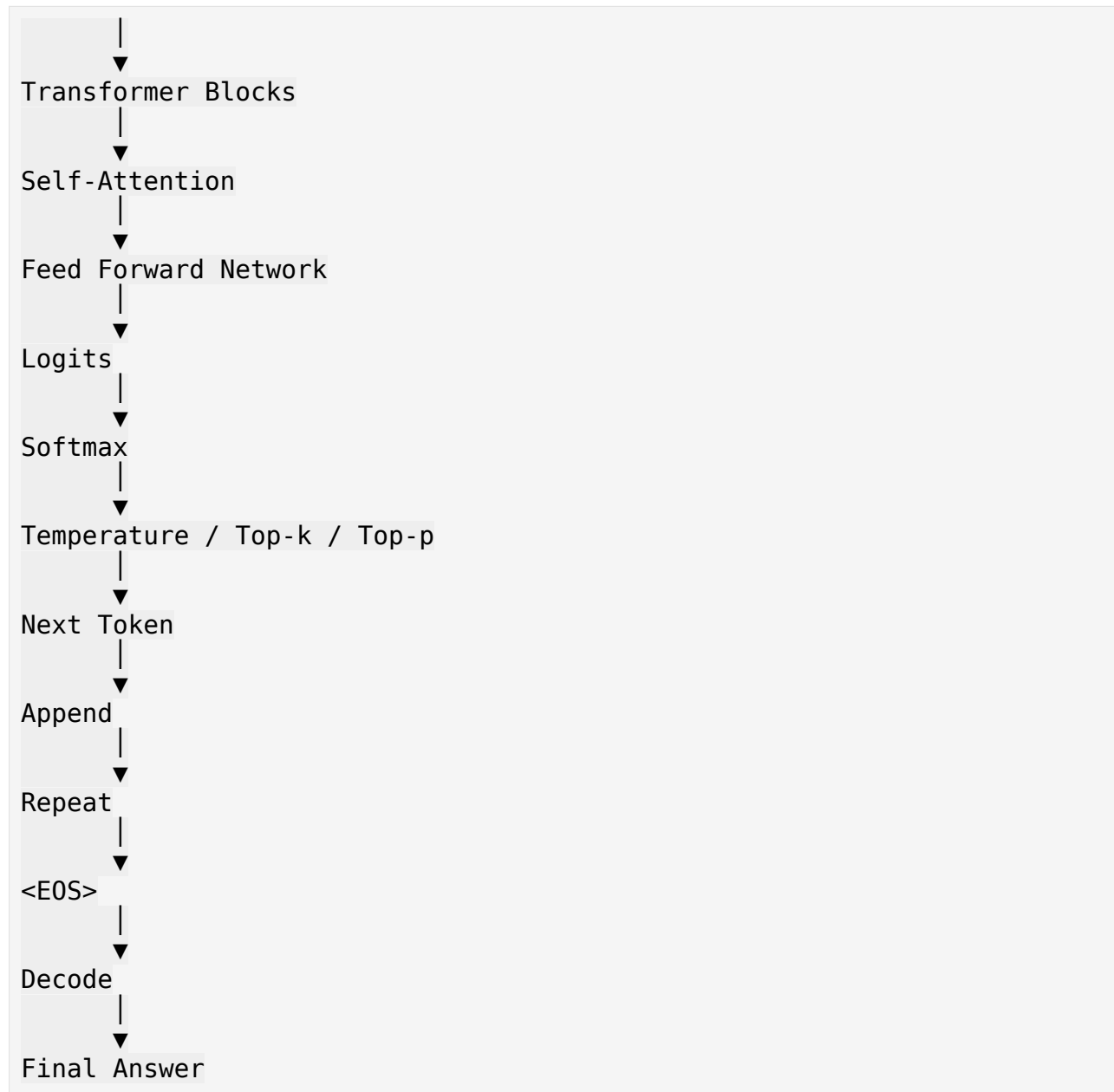
Yes.

Reasons include:

- Ambiguous prompts
 - Incorrect training patterns
 - Limited context
 - Probabilistic generation
 - Hallucinations (producing plausible but incorrect information)
-

12.22 Complete Master Pipeline





12.23 Interview Questions

Q1. How does ChatGPT answer a question?

Answer: It tokenizes the input, converts tokens into embeddings, processes them through Transformer layers with self-attention, computes logits, converts them into probabilities using Softmax, selects the next token using a decoding strategy, appends it to the sequence, and repeats until completion.

Q2. Does ChatGPT generate an entire paragraph in one step?

Answer: No. It generates **one token at a time** using autoregressive decoding.

Q3. What happens after tokenization?

Answer: Tokens are mapped to token IDs, transformed into embeddings, and processed by Transformer layers.

Q4. What is the role of Softmax?

Answer: Softmax converts raw logits into a probability distribution over possible next tokens.

Q5. Does inference change the model's weights?

Answer: No. During inference, the model performs forward computation only and does not update its parameters.

❑ Common Mistakes

"ChatGPT searches the internet for every answer."

Not necessarily. A base LLM generates responses from its learned parameters unless explicitly connected to external tools.

"ChatGPT predicts whole sentences."

Wrong.

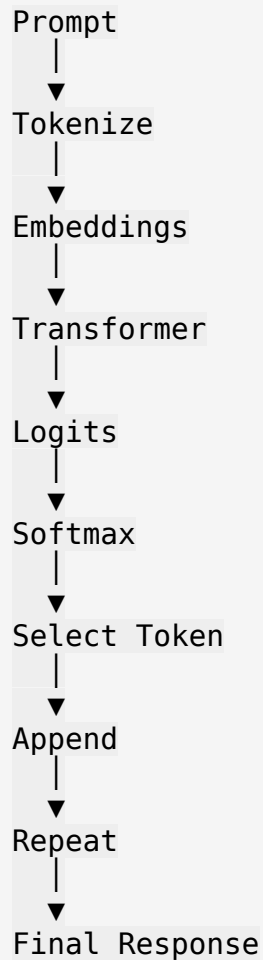
It predicts **one token at a time**.

"Temperature changes what the model knows."

Wrong.

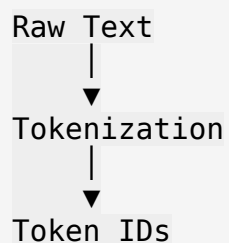
It changes **how tokens are sampled**, not the knowledge stored in the model.

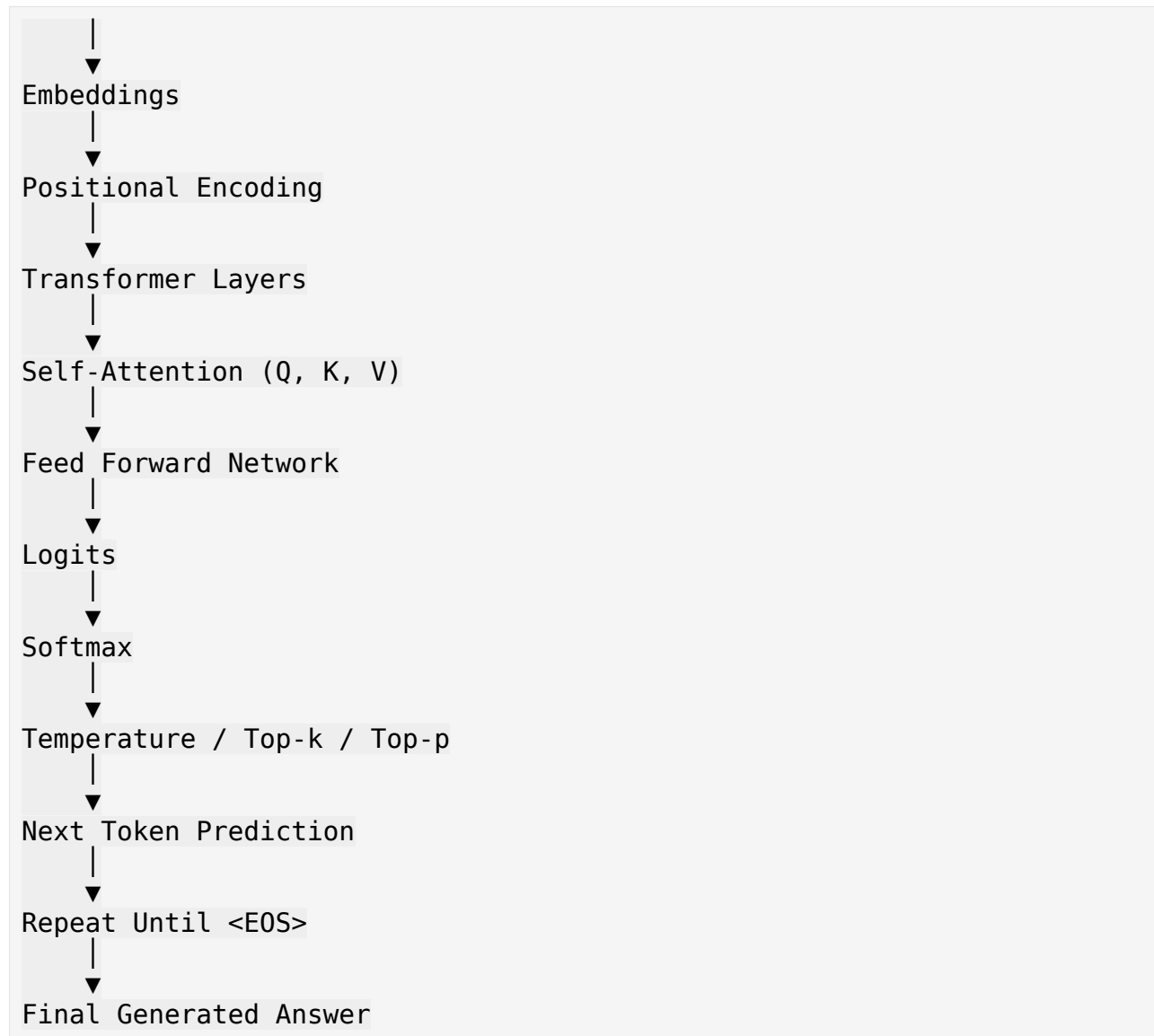
□ Memory Trick



□ Final LLM Interview Summary (Parts 1–12)

If you remember just this sequence, you can explain the entire LLM pipeline in an interview:





□ Ultimate Interview Answer (30 Seconds)

"A Large Language Model first tokenizes the user's input and converts it into token IDs. These IDs are mapped to embeddings and enriched with positional information. The embeddings pass through multiple Transformer layers, where self-attention enables each token to interact with others using Query, Key, and Value vectors. The model produces logits, Softmax converts them into probabilities, and a decoding strategy such as temperature or top-k selects the next token. This process repeats one token at a time until an end-of-sequence token is reached, producing the final response."

🎉 Congratulations!

After completing **Parts 1–12**, you now have a **strong conceptual foundation in LLMs**, covering:

- 📖 What LLMs are
- 📖 Why traditional programming evolved into LLMs
- 📖 Tokens and tokenization
- 📖 Embeddings
- 📖 Why embeddings matter
- 📖 Context windows
- 📖 Transformer architecture
- 📖 Attention and Self-Attention
- 📖 Training phase
- 📖 Inference phase
- 📖 Temperature, Top-k, and Top-p
- 📖 End-to-end response generation in ChatGPT

These notes provide an excellent base for **AI Engineer, Machine Learning Engineer, and Generative AI interview preparation. bold text**